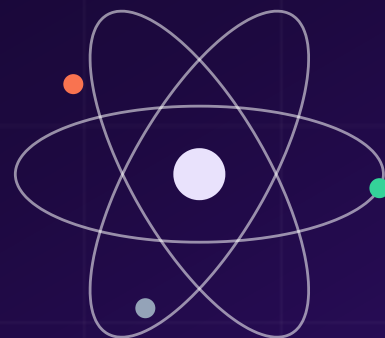




E S P E C I F I C A C A O   T E C N I C A   E   A R Q U I T E T U R A L   M A S T E R

# Axión

V 0 . 2

Linguagem Funcional de Sistemas — Estrita, Linear, Segura e com Latência Zero

Tipos Lineares %1

Sem Garbage Collector

Ternário Balanceado

Topologia de Tipos de Custo Zero

Compilador e Runtime de Raiz

# Sumário

|    |                                                       |
|----|-------------------------------------------------------|
| -  | Sumário Executivo (TL;DR)                             |
| 00 | Análise Crítica e Guia de Leitura                     |
| 01 | Visão Geral e Filosofia                               |
| 02 | Gestão de Memória e Automação da Linearidade          |
| 03 | Avaliação Híbrida e Co-Propriedade de Dados           |
| 04 | Mutação Local Encapsulada                             |
| 05 | Interoperabilidade Nativa Segura (FFI Linear)         |
| 06 | Concorrência Espacial: Canais e Session Types         |
| 07 | Erros e Pânicos Sem Fugas (Linear Unwinding)          |
| 08 | Compilador: Velocidade e Ergonomia (Salsa)            |
| 09 | Paradigma de Design: Topologia de Tipos               |
| 10 | O Tipo Trit e a Compactação Ternária                  |
| 11 | Pipeline de Compilação Consolidado                    |
| 12 | Gramática (EBNF)                                      |
| 13 | Grandes Quadros Comparativos de Engenharia            |
| 14 | Análise Crítica e Objecções de Mercado (O Fogo Amigo) |
| 15 | Apêndice: Riscos, Trade-offs e Trabalho Futuro        |
| 16 | Referências e Prior Art                               |
| 17 | Plano de Implementação (do Início ao Fim)             |
| 18 | Decisões de Implementação (Stack e Auto-Hospedagem)   |
| 19 | Exemplo Completo: um Módulo de Ponta a Ponta          |

A *Axió*n numa página — antes da densidade que se segue.

### O que é

Uma linguagem funcional de sistemas: sintaxe de Haskell estrito, **sem GC**, com posse e tipos lineares *inferidos* (sem lifetimes explícitos), regiões/arenas, session types e latência zero.

### Para quem

Nichos onde o jitter do GC é inaceitável e a segurança é crítica: HFT, kernels, motores de crypto, tempo-real, drivers. **Não** é de propósito geral (§1, Não-objetivos).

## Os três argumentos

- 1 Segurança de Haskell, controlo de C.** Sem uso-após-livre, sem data races, sem fugas — provado por tipos, não por convenção; e sem GC, com libertação em pontos estáticos exatos (Auto-Drop, §2).
- 2 Linearidade sem a luta do borrow checker.** A posse e as regiões são inferidas; o compilador Socrático mostra o grafo e sugere a correção. Divulgação progressiva L0-L3: 90% do código nunca vê linearidade (§8).
- 3 Feedback sub-milissegundo.** Compilador reativo (Salsa), backend instantâneo em dev (Cranelift) e otimizado em release (LLVM) (§8, §11).

**Estado (honesto).** Este documento fixa a *visão arquitetural*, não uma implementação. O núcleo (linear + memória + session types) é engenharia estabelecida (§16); a metateoria mecanizada, os benchmarks e as camadas opcionais (compactação ternária, topologia avançada) estão *por provar* e assim marcados. Ver §14 (fogo amigo) e §17 (plano).

**Percursos de leitura.** Céptico de sistemas → §13 (quadros), §14 (objecções), §16 (posicionamento). Curioso de FP → §1, §2, §9. Implementador → §11, §12, §17, §18, §19 (exemplo completo).

*Este documento fixa a visão arquitetural da Axión (V0.2), clarifica cada mecanismo com intuição e diagramas, e separa honestamente o que é engenharia estabelecida do que é investigação em aberto.*

A Axión combina três linhagens reais e bem fundamentadas:

- **Tipos lineares e de unicidade** — a base teórica existe (lógica linear de Girard, tipos de unicidade da linguagem Clean, e a extensão `LinearTypes` do GHC com a multiplicidade `%1`). A gestão de memória sem GC por regiões/arenas é território conhecido (Cyclone, MLKit, Rust).
- **Ternário balanceado** — um sistema numérico real (o computador soviético Setun, 1958) com uma propriedade elegante de compactação:  $3^5 = 243 \leq 256$ , logo cinco trits cabem num byte.
- **Compilador reativo** — o motor de consultas «Salsa» é a mesma arquitetura que sustenta o `rust-analyzer`; a incrementalidade sub-milissegundo é plausível.

**O que está sólido:** linearidade + Auto-Drop, arenas com reset não-léxico, Stream Fusion, defuncionalização de `Susp`, FFI com `@cleanup`, canais com session types, e o tipo `Trit` (um enum de três estados para `observe` / Kleene). Tudo isto é implementável com técnicas conhecidas. **Opcional e por medir:** a compactação ternária base-243 não é engenharia estabelecida — é uma otimização de biblioteca (`TritVec`), desligada por omissão e condicionada a benchmarks (§10, §14 #07).

**Concorrência e a topologia de tipos:** os operadores `~` e `Maybe~` são açúcar sintático que *desugar* para os canais lineares da §06 — o efeito «não-local» entre threads é sempre mediado por um canal linear, com a posse a mover-se em vez de ser partilhada. Por isso «zero data races» é um corolário da Fidelidade de Sessão. `&` é a **conjunção aditiva** da lógica linear (escolhe-se um de dois potenciais) e não há leitura literal de «computação quântica»: não há qubits, há uma **topologia de tipos** inspirada na mecânica quântica. A ausência de deadlocks vem do bloco `bound` — um *nursery* de concorrência estruturada cujos canais formam uma árvore (topologia de comunicação acíclica por construção).

**Espectro de permissões** (`%1`, `%0.5`): a multiplicidade qualifica o tipo de um recurso e determina o acesso. `%1` é posse plena — leitura e escrita exclusivas; `%0.5` é uma permissão fracionária ao estilo de Boyland — metades que concedem leitura partilhada e recombinaem em `%1` para recuperar a escrita (ver §2). A Elisão de Empréstimos é a forma implícita deste modelo; `%0.5`, a forma explícita e de primeira classe.

**Nota normativa — relação com o GHC e o Haskell.** A Axión não é um dialecto do Haskell nem corre sobre o GHC: é uma **reescrita completa** — parser, typechecker, IR, backend e runtime próprios. O GC e a laziness do GHC são estruturalmente incompatíveis com as garantias da linguagem (latência zero, arenas, Linear Unwinding), pelo que **não há, por decisão de design, compatibilidade com código ou bibliotecas Haskell existentes**: as bibliotecas do Hackage pressupõem um runtime lazy com GC e não linkam com o runtime da Axión. Do Haskell herda-se a sintaxe, o modelo mental e a linhagem teórica (Linear Haskell, POPL 2018); a estratégia de ecossistema é a FFI Linear para C e Rust (§05). O Linear Haskell serve ainda de bancada de validação semântica no protótipo descartável da Fase 0 (§17).

## 01 Visão Geral e Filosofia

A Axión é uma linguagem funcional pura de sistemas — uma **reescrita de raiz da linhagem do GHC** (Glasgow Haskell Compiler): compilador, backend e runtime próprios, desenhados desde a primeira linha para uma semântica estrita, linear e sem GC. Do Haskell herda a sintaxe e o modelo mental; **não herda o runtime, a compatibilidade binária, nem o ecossistema Hackage**. A premissa fundamental é **unificar dois mundos** que raramente coexistem:

### Do Haskell

Elegância matemática, minimalismo sintático, pureza e desacoplamento lógico.

### Do Rust e do C

Determinismo de memória, latência zero e controlo direto do hardware.

Para o conseguir, a Axión **elimina por completo o Garbage Collector**, abolindo o seu *overhead* e as pausas de execução ( `jitter` ). No lugar do GC entra um paradigma de **posse (ownership) e tipos lineares ( %1 )**: o compilador rastreia estaticamente o ciclo de vida de cada dado e automatiza a sua libertação — sem as anotações manuais de tempo de vida (*lifetimes*) que caracterizam o Rust.

**Intuição:** onde o Rust exige lifetimes explícitos ( `<'a>` ) quando as regras de elisão não chegam, a Axión *infe*re a região. Onde o Haskell paga um GC, a Axión insere `free` em pontos exatos determinados por análise de fluxo.

## Primeiros Exemplos (nível L0)

Antes de qualquer mecanismo avançado, eis a linguagem no nível **L0** — sintaxe e semântica familiares de um Haskell estrito, **sem uma única anotação de linearidade**. Quem sabe programação funcional já lê isto no dia 1.

Listagem 1.1 – Olá Mundo

```
main :: IO ()
main = putStrLn "Hello, Axión!"
```

Listagem 1.2 – Fibonacci: recursão e o «loop» funcional (acumulador)

```
-- Recursive definition: pattern matching + recursion.
fib :: Int -> Int
fib 0 = 0
fib 1 = 1
fib n = fib (n - 1) + fib (n - 2)

-- Fast iterative version: an accumulator is a functional "loop" (O(n)).
fibFast :: Int -> Int
fibFast n = go n 0 1
  where
    go 0 a _ = a
    go k a b = go (k - 1) b (a + b)
```

### Listagem 1.3 – FizzBuzz: guardas, operador mod e iteração 1..N

```
-- Guards = an if/else chain; `mod` is the remainder operator (%).
fizzbuzz :: Int -> String
fizzbuzz n
  | n `mod` 15 == 0 = "FizzBuzz"
  | n `mod` 3  == 0 = "Fizz"
  | n `mod` 5  == 0 = "Buzz"
  | otherwise      = show n

-- Loop over 1..100 (a strict range; Stream Fusion, no heap allocation).
main :: IO ()
main = mapM_ (putStrLn . fizzbuzz) [1 .. 100]
```

### Listagem 1.4 – Fatorial: funções e tipos numéricos (Int vs Integer / overflow)

```
-- 'Int' is fixed-width and WOULD overflow for large n.
-- 'Integer' is arbitrary-precision: no overflow, at a small runtime cost.
factorial :: Integer -> Integer
factorial 0 = 1
factorial n = n * factorial (n - 1)

-- factorial 20 fits in an Int; factorial 50 needs Integer.
```

## Mapa dos mecanismos (a «topologia»)

Tudo deriva de uma disciplina subestrutural única; as peças encaixam assim — e é este grafo que o compilador infere, mostra e chama «topologia de tipos»:



Atravessam tudo: Auto-Drop (§2), Linear Unwinding (§7) e a divulgação progressiva L0-L3 (§8). Cada peça deriva da mesma regra linear.

## Não-objetivos (o que a Axióm *não* é)

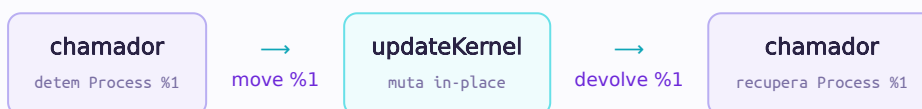
Tão importante como o que promete é o que recusa fazer — declarado à cabeça, porque uma linguagem que sabe onde não vai é mais fiável:

- **Não é de propósito geral.** É de nicho, alto desempenho. Para prototipagem rápida, scripts ou a maioria do software de aplicação, Python/Go/TypeScript continuam a ser a escolha certa.
- **Não troca segurança por conveniência.** Sem GC, não há a rede que o esconde; a disciplina linear é obrigatória em L1+.
- **Não é compatível com o Haskell.** Sem Hackage, sem runtime lazy, sem código GHC reutilizável — herda a sintaxe e a linhagem teórica, não o ecossistema (§0, §16).
- **FFI não é seguro por omissão.** O `@cleanup` governa fugas; contra escritas selvagens de C, o isolamento é **opt-in e em camadas** ( `@isolate(mpk)` / `@sandbox(wasm)` , §5) — declarado por biblioteca, não mágico nem grátis. O default ( `@trusted` ) confia, como o `unsafe` FFI do Rust.

- **Não promete os números ainda.** As garantias de performance são desenho, não medição — ficam sob benchmark (§13).

O pipeline executa três análises estáticas que, em conjunto, eliminam cópias ocultas e anotações manuais de ciclo de vida:

- 1 **Elisão de Empréstimos** (*Implicit Borrowing*) — quando uma função apenas *lê* um dado imutável sem o consumir, o verificador converte o tipo linear numa vista de baixo nível ( `slice` ). A posse regressa ao escopo original de forma invisível.
- 2 **Destruição Automática** (*Auto-Drop*) — uma análise de vivacidade (*Liveness*) identifica o ponto exato em que uma variável morre e injeta aí a instrução nativa `free`.
- 3 **Atualização Destrutiva In-Place** (*Linear Elision*) — se um recurso é linear ( `%1` ) e a atualização de registo é a sua última menção viva, o compilador transmuta o código funcional numa mutação imperativa direta.



A posse é transferida estaticamente: nunca há duas referências vivas ao mesmo buffer.

Fig. 2.1 — Transferência de posse: a linearidade garante uma única referência viva.

#### Listagem 2.1 — Auto-Drop e mutação in-place inferidos pelo compilador

```
data Process = Process
{ pid    :: Int
, status :: String
, buffer :: Buffer U8 %1      -- linear field embedded in the record
}

-- 'p' is consumed (%1) and returned (%1): ownership goes in and out, never cloned.
updateKernel :: Process %1 -> Process %1
updateKernel p =
  let p' = p { status = "Running" }  -- last live mention of 'p'
  in p'
  -- 1. The inner buffer is never copied (borrow elision).
  -- 2. 'p' dies here -> the compiler MUTATES the 'status' field in place.
  -- 3. If 'p' were not returned, Auto-Drop would inject free(p'.buffer).
```



## Listagem 2.2 – Elisão de Empréstimos: ler sem consumir

```
-- 'checksum' only READS the buffer: no '%1' on its argument => the compiler
-- borrows it as a low-level view (slice); ownership never leaves the caller.
checksum :: Buffer U8 %1 -> U32          -- borrow: no '%1', so 'buf' is NOT consumed
checksum buf = foldBytes (+) 0 buf

encrypt :: Buffer U8 %1 -> Buffer U8 %1  -- consumes and returns ownership
encrypt buf = imperative $ do ...

process :: Buffer U8 %1 -> (U32, Buffer U8 %1)
process buf =
  let sig = checksum buf      -- implicit borrow: 'buf' is STILL owned after this line
  in (sig, encrypt buf)      -- ownership flows on into 'encrypt'; no clone, no AX0001
```

A regra da linearidade (destruidor como consumidor). Todo o valor `%1` é consumido **exatamente uma vez** — pelo programador, ou pelo seu destruidor (`Drop`). A contração (usar duas vezes) é proibida para todos os `%1`, sempre; o enfraquecimento (descartar) só é permitido para tipos com instância `Drop`, e o Auto-Drop materializa-o como a chamada ao destruidor no ponto de morte. Tipos sem `Drop` — endpoints de sessão (`Ep`), `Token`, handles de transação — são *must-use*: esquecê-los é erro de compilação (é disto que depende a Fidelidade de Sessão da §9). `Drop` propaga estruturalmente: um registo é droppable se todos os campos o forem; uma closure (e um `Susp`) herda a obrigação do que captura. O `@cleanup` da §5 é a instância `Drop` de um `LinearPtr`, declarada na importação.

## Permissões Fracionárias ( `%0.5` )

A multiplicidade de um recurso exprime *quem lhe acede*. `%1` é posse plena — leitura e escrita exclusivas. `%0.5` é uma **permissão fracionária** ao estilo de Boyland: um `%1` divide-se (`split`) em metades que concedem leitura partilhada (nunca escrita) e recombina-se (`join`) em `%1` para recuperar a escrita e a posse. A Elisão de Empréstimos é a forma implícita deste modelo; `%0.5` é a sua forma explícita e de primeira classe. O Auto-Drop só liberta o recurso quando as metades voltam a formar `%1` ou quando todas morrem.

## Listagem 2.3 – Split/join de permissões fracionárias

```
-- 'split' divides %1 into two shared-read halves.
let (a, b) = split cfg      -- cfg : Config %1 => a, b : Config %0.5
readOnly a                  -- safe concurrent reads; neither one writes
readOnly b
let cfg' = join a b         -- recombines into Config %1; write access returns
-- Auto-Drop of 'cfg' only fires after the join (or once both halves die).
```

## Listagem 2.4 – Um recurso must-use esquecido (AX0002)

```
error[AX0002]: recurso linear sem Drop descartado sem ser consumido
--> src/Session.axi:31:5
|
28 |   ep <- openChannel svc
|   --- 'ep' : Ep %1 (Send Request End) – endpoint de sessão (sem Drop)
|
31 |   pure result
|   ^^^^^^^^^ 'ep' ainda está vivo aqui e nunca foi consumido
|
= porquê: endpoints, Token e handles de transação são must-use – não têm
  instância Drop, logo esquecê-los é erro, não uma fuga silenciosa (§2).
= ajuda [machine-applicable – o LSP aplica com um clique]:
  consome a sessão antes de devolver:
    30 |     sentEp <- send ep request
    31 |     close sentEp
= explicação completa: axion --explain AX0002
```

## A. Preguiça Linear Dirigida ( `Susp` )

A Axió é **estrita por omissão**, com preguiça *explícita e validada* através do tipo primitivo `Susp a` — um fecho (*closure*) que encapsula uma computação adiada. Sendo linear, a suspensão obedece à regra da §2: **tem de ser consumida exatamente uma vez** — por `force`, ou pelo destruidor se todas as capturas forem droppable; um `Susp` que capture um endpoint herda o *must-use*. Ao ser disparada por `force`, os seus recursos intermediários são limpos pelo Auto-Drop.

**Otimização de unboxing:** em `Release`, a *defuncionalização estática* compacta as variáveis capturadas numa estrutura contígua na Stack (ou arena local), convertendo a chamada dinâmica num salto de hardware direto — sem indireção de ponteiros.

Listagem 3.1 – `Susp`: computação adiada com custo zero de runtime

```
-- Heavy deferred computation, with zero runtime cost.
generateCryptoKey :: Buffer U8 %1 -> Susp (Buffer U8 %1)
generateCryptoKey buf = susp $ do
    sha256Algorithm buf      -- 'buf' lives directly on the Stack (unboxing)

-- Deterministic consumption: 'force' fires exactly once.
processSecurity :: Susp (Buffer U8 %1) %1 -> Buffer U8 %1
processSecurity suspKey =
    let key = force suspKey    -- direct jump, no pointer indirection
    in key
    -- Auto-Drop sweeps the suspension allocation right after the 'force'.
```

## B. Linear Streams para Processamento Infinito

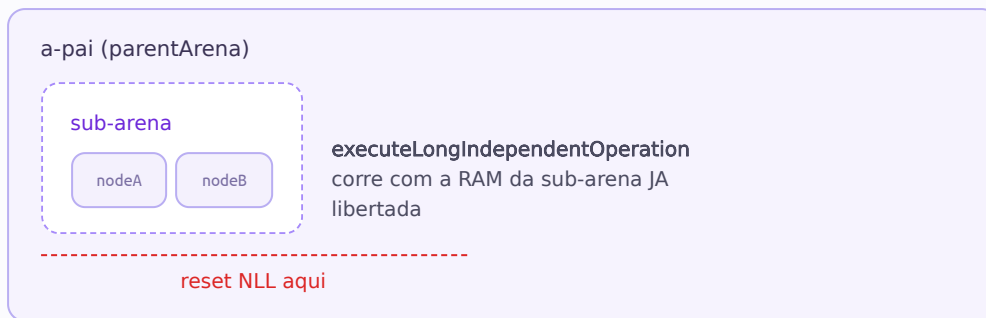
Com **Stream Fusion**, cadeias de transformações sobre fluxos contínuos colapsam num único *loop* imperativo nativo, sem alocação extra na Heap.

Listagem 3.2 – Pipeline de pacotes fundido num único loop

```
-- "Infinite" network stream with 0 extra bytes on the Heap.
filterPackets :: LinearStream Packet %1 -> LinearStream Packet %1
filterPackets stream =
    stream
    |> stream_filter (\p -> packet_port p == 443)
    |> stream_map    (\p -> apply_decryption p)
    -- Stream Fusion collapses it all into a single 'while' loop in native Assembly.
```

## C. Arenas Hierárquicas com Reset Não-Léxico (NLL)

As regiões de memória expandem-se e contraem-se dinamicamente. Os dados temporários são eliminados no instante em que a última célula dependente sai de uso — **ignorando os limites de indentação léxica**.



Liveness deteta a morte de nodeA/nodeB antes do fim lexico -> ponteiro da sub-arena recua numa unica instrucao.

Fig. 3.1 — O reset da sub-arena é injetado no ponto de morte real, não no fim do bloco.

#### Listagem 3.3 — Sub-arena descartável com reset antecipado

```
buildTemporaryGraph :: Arena %1 -> IO ()
buildTemporaryGraph parentArena = withSubArena parentArena $ \subArena -> do
  let nodeA = allocateCell subArena "Node A"
  let nodeB = allocateCell subArena "Node B"
  linkNodes nodeA nodeB
  processGraphData nodeA
  -- Liveness detects that nodeA/nodeB die on this line.
  -- The sub-arena reset is injected HERE (not at the lexical end of the block):
  executeLongIndependentOperation -- already runs with the sub-arena RAM cleared
```

## Escape estático é erro de compilação ( `promote` )

A mesma análise de *liveness* que injeta o reset NLL deteta valores cuja vida ultrapassa o reset da sub-arena. Um valor que *escapa* (por retorno ou captura em closure) é **rejeitado pelo typechecker** — nunca copiado em silêncio. Para o manter vivo, move-se explicitamente para a arena-pai com `promote`: o custo fica visível e a retenção por escape torna-se impossível por construção.

#### Listagem 3.4 — 'promote' move um valor para a arena-pai antes do reset

```
withSubArena parentArena $ \subArena -> do
  let node = allocateCell subArena "alive after reset"
  -- 'node' would die at the subArena reset: the typechecker demands promotion.
  let node' = promote parentArena node -- moves to the parent arena; survives
  subArena_reset -- reclaims the rest; 'node' stays valid
  node'
```

#### Listagem 3.5 – Escape de sub-arena é erro de compilação (AX0003)

```
error[AX0003]: um valor escapa da sua sub-arena
--> src/Graph.axi:12:5
|
10 |     let node = allocateCell subArena "escapes"
|         --- 'node' vive na sub-arena 'subArena'
|
12 |     node
|     ^^^^ devolvido daqui – sobreviveria ao reset da sub-arena
|
= porquê: no reset a RAM da sub-arena é recuperada; um valor que escapa
ficaria pendurado. O escape é rejeitado, nunca copiado em silêncio (§3C).
= ajuda [machine-applicable – o LSP aplica com um clique]:
move-o para a arena-pai antes do reset:
11 |     let node' = promote parentArena node
12 |     node'
= sem cópia? em L1+, box node (RC determinístico, §3) – ou &? para só espreitar
= explicação completa: axion --explain AX0003
```

### Marcas de arena ( `arena_mark` / `arena_release` )

Para reclamar memória *dentro* de uma arena sem criar uma sub-arena, uma **marca** guarda o topo do *bump pointer* e o `arena_release` recua-o — reclamação intra-escopo ao custo de uma instrução. Tudo o que é alocado depois da marca é consumido linearmente ou libertado no release.

#### Listagem 3.6 – Reclamação intra-escopo por marca/release

```
let mark = arena_mark arena      -- saves the top of the bump pointer
let tmp = allocateCell arena "temporary"
process tmp
arena_release mark               -- rewinds the pointer; reclaims without a sub-arena
```

### Decommit no reset

No reset, as páginas acima de um limiar são devolvidas ao sistema operativo ( `madvise(DONTNEED)` ou equivalente), pelo que o *high-water mark* não permanece residente. Não altera a correção — reduz a pegada de RAM sob picos de alocação.

### Escape-hatch opt-in: `Boxed a` (RC determinístico, estilo Perceus)

Para a minoria de valores cuja vida não cabe na disciplina de regiões, o tipo opt-in `Boxed a` oferece *reference counting* preciso e determinístico (Perceus, PLDI 2021): sem GC, sem pausas, custo visível no tipo. A análise de linearidade já dá os pontos de drop estaticamente, pelo que a maioria das operações de RC se elide em compilação; e um `Boxed` imutável **não pode formar ciclos** (a mutação interior vive nos Tokens Fantasma, §3D) — logo não é preciso coletor de ciclos. Honestidade: o drop de uma estrutura grande é uma cascata  $O(n)$  — determinística e escalonável, não «custo zero». Guardas contra a erosão da disciplina: `Boxed` é L1+ e um lint conta os `Boxed` por módulo — o caminho fácil não pode vencer o caminho certo. Estado: definido; implementação pós-Fase 2, condicionada à telemetria de erros (§8) — se o escape não for a fricção prevista, o subsistema não se constrói. API e uso na Listagem 3.7.

#### Listagem 3.7 – Boxed: a API e um escape sem promote

```
-- 'Boxed a' TEM instância Drop – o decref – logo o Auto-Drop gere os handles;
-- não existe 'free' de Boxed. A análise de linearidade elide a maioria dos incr/decr.
box  :: a %1 -> Boxed a          -- um único move para a heap RC
share :: Boxed a %1 -> (Boxed a, Boxed a) -- dois donos (incr, muitas vezes elidido)
-- Leituras usam a elisão de empréstimos normal ($2): espreitar um Boxed
-- nunca toca no contador.

-- Uso: uma config escapa da sub-arena SEM promote (sem cópia para a arena-pai):
withSubArena parentArena $ \subArena -> do
  let cfg = buildConfig subArena -- morreria no reset (AX0003)
  let bcfg = box cfg             -- um move para a heap RC; só em L1+
  let (a, b) = share bcfg        -- posse partilhada, determinística
  spawnLogger a                  -- vive enquanto algum handle viver
  pure b                         -- Auto-Drop faz decref quando cada handle morre
```

## D. Tokens Fantasma de Co-Propriedade e o bloco `using`

Para modelar grafos cíclicos ou tabelas interligadas na Heap sem abdicar da segurança linear, os **Tokens Fantasma** separam a topologia dos dados da sua permissão de mutação. O bloco `using` oculta o encadeamento manual do token.

#### Listagem 3.8 – Mutação de grafo sob um único token de controlo

```
-- Phantom Tokens separate the data TOPOLOGY from the mutation PERMISSION.
updateTopology :: Token s %1 -> Cell s Server -> Cell s Server -> Token s %1
updateTopology token s1 s2 = using token $ do
  mutate s1 (\srv -> srv { connections = connections srv + 1 })
  mutate s2 (\srv -> srv { active = True })
-- 'using' returns the token at the end; the manual threading stays hidden.
```

O bloco `imperative` desbloqueia uma região isolada de fronteiras puras, desenhada para computação numérica pesada e manipulação contígua de buffers com vetorização nativa (AVX/SSE). Por fora, a função continua a ser pura e referencialmente transparente.

Listagem 4.1 – Escalonamento de vetor com SIMD e loop unrolling

```
-- Isolated imperative region, with pure boundaries and native vectorization.
blasScale :: F64 -> Buffer F64 %1 -> Buffer F64 %1
blasScale factor vec = imperative $ do
  let len = buffer_length vec
  for 0 len $ \i -> do
    let v = mut_read vec i
    mut_write vec i (v * factor)
  vec
-- Loop Unrolling + packed SIMD (AVX/SSE) instructions.
```

## 05 Interoperabilidade Nativa Segura (FFI Linear Autogovernada)

A Axióon integra ponteiros brutos de bibliotecas externas (C/Rust) no motor de inferência de linearidade através do tipo primitivo `LinearPtr a`, associando funções de limpeza diretamente na importação via `@cleanup`. O programador nunca escreve boilerplate de libertação.

Listagem 5.1 – Recurso OpenSSL com cleanup injetado pelo compilador

```
-- FFI with automatic lifecycle governance via @cleanup.
foreign import ccall "openssl_ctx_free" @cleanup
  freeSSL :: LinearPtr SSL_CTX -> IO ()

foreign import ccall "openssl_ctx_init"
  initSSL :: IO (LinearPtr SSL_CTX)

executeTransaction :: String -> IO ()
executeTransaction payload = do
  ctx <- initSSL           -- allocated in C's memory space
  sendSecureData ctx payload
  -- No boilerplate: the compiler injects openssl_ctx_free(ctx)
  -- as soon as 'ctx' is no longer live.
```

Listagem 5.2 – A fronteira de confiança: `trusted`, `assume` e o efeito `Raw`

```
-- Sem 'unsafe': o acesso cru é um EFEITO visível no tipo de retorno ('Raw'),
-- e a fronteira de confiança irreduzível marca-se com 'trusted' (a TCB auditada).
readDevice :: LinearPtr MMIO -> Int -> Raw U32  -- 'Raw': lê memória mapeada
readDevice reg off = trusted $ do              -- bloco auditado; entra na TCB
  peekOffU32 reg off

-- 'assume' descarrega UM facto pontual que o compilador não prova,
-- com justificação obrigatória (fica no registo de auditoria).
alignedLoad :: LinearPtr F64 %1 -> Vec4 F64
alignedLoad p =
  assume (aligned64 p) "libc garante malloc alinhado a 64 bytes" $
    loadVectorAligned p
-- Fora de 'trusted'/'assume' tudo é verificado; estes são os ÚNICOS pontos de confiança.
```

**Modelo de confiança (sem `unsafe`):** a Axióon não tem blocos `unsafe` ao estilo do Rust. O que o Rust confia por asserção decompõe-se em três: operações com prova (descarregam uma obrigação verificada — sem etiqueta, porque são seguras); acesso cru rastreado por capacidade (efeito `Foreign` / `Raw` visível no tipo); e a fronteira de confiança irreduzível, marcada honestamente por `trusted` (a TCB que se audita) ou `assume` para um facto pontual. A chamada `foreign` corre numa arena dedicada, o que confina os modos de falha bem-comportados — fugas e frees esquecidos (o `@cleanup` é a instância `Drop` do `LinearPtr`, §2) — e limpa o estado estrangeiro num reset. Honestidade: um ponteiro selvagem em C pode escrever em qualquer endereço do processo — e por isso o isolamento é uma **política em camadas, opcional e declarada no `foreign`** (abaixo); o default é a arena auditada ( `trusted` ).

### Isolamento em camadas (opt-in, declarado no `foreign`)

A arena dedicada confina *bugs* — uma escrita accidental fora do buffer fica na arena — mas não um ponteiro que o C *fabrique* de propósito. Por isso o isolamento real é uma **política escolhida por biblioteca**, visível na



declaração e com o custo à vista. Dois modelos de ameaça, respostas distintas:

| Anotação       | Mecanismo                                                                         | Custo                                            | Protege contra                                   |
|----------------|-----------------------------------------------------------------------------------|--------------------------------------------------|--------------------------------------------------|
| @trusted       | TCB auditada, sem isolamento (o default)                                          | Zero                                             | — (confia-se; equivale ao unsafe FFI do Rust)    |
| @isolate(mpk)  | Memory Protection Keys: heap da Axióon e arena foreign em domínios PKRU distintos | ~dezenas de ciclos/chamada; sem recompilar a lib | Escritas selvagens na heap (x86; ARM: MTE)       |
| @sandbox(wasm) | SFI: a lib compila para WASM ( wasm2c ) e corre in-process com memória própria    | Bounds checks + cópia na fronteira (moderado)    | C malicioso — portátil, sem hardware especial    |
| (CHERI)        | Capabilities com limites em hardware (Arm Morello) — plataforma, não anotação     | ~zero em hardware CHERI                          | Endgame: segurança espacial imposta pelo silício |

Listagem 5.3 – Isolamento escolhido por biblioteca

```
-- C não-confiável (parser de imagem): isolado num sandbox WASM.
foreign import ccall "png_read" @sandbox(wasm) readPng :: LinearPtr Bytes %1 -> IO (Image %1)

-- Confia-se, mas protegem-se as páginas da Axióon durante a chamada (~1 write ao PKRU).
foreign import ccall "fast_memcpy" @isolate(mpk) memcpy :: LinearPtr Bytes %1 -> LinearPtr Bytes -> Int -> IO ()

-- TCB auditada: velocidade máxima, sem isolamento (o default).
foreign import ccall "malloc" @trusted cmalloc :: Int -> IO (LinearPtr Bytes)

-- Sem anotação de isolamento => @trusted implícito. A escolha (e o custo) ficam à vista no código.
```

**A regra de ouro.** O default é rápido e auditado ( @trusted ); o isolamento é um botão que se liga por biblioteca, pagando o custo conscientemente. A Axióon não promete «FFI seguro por magia» — promete uma **fronteira de confiança explícita, mínima e escalável**: de auditoria ( @trusted ) a páginas por hardware ( @isolate ) a sandbox portátil ( @sandbox ) a capabilities (CHERI). A arena confina bugs; MPK/WASM confinam ataques. Prior art: RLBox (USENIX Security 2020), ERIM/Hodor (MPK), CHERI/Morello.

Para um ambiente multi-core livre de **data races** sem o custo de *locks*, mutexes ou barreiras atômicas em runtime, a Axióon adota **Canais Lineares** governados por **Session Types** ao nível do compilador. Quando uma thread transfere dados, a posse física do endereço é *movida estaticamente*: a thread remetente fica proibida pelo typechecker de tocar no ponteiro a partir do envio. Os canais garantem *race-freedom*; a *deadlock-freedom* por construção (concorrência estruturada) surge na §9.



Fig. 6.1 — Handoff linear: o remetente congela, o recetor recebe posse exclusiva.

Listagem 6.1 – Protocolo tipado: envia um buffer e termina

```
-- Session Type: the channel sends ONE linear Buffer and then ends (End).
type CryptoService = Ep %1 (Send (Buffer U8 %1) End)

workerThread :: Channel CryptoService %1 -> IO ()
workerThread chan = do
  let buf = allocateLocalBuffer 4096
  sentChan <- sendData chan buf -- 'buf' AND 'chan' are consumed here
  closeChannel sentChan
  -- Any later read of 'buf' or 'chan' is REJECTED at compile time.
```

Em linguagens de sistemas sem GC, um pânico costuma provocar fugas de memória. A Axióon resolve-o com o **Linear Unwinding**: quando ocorre um pânico, o runtime executa um salto estruturado para o limite da sub-arena mais próxima na Stack.

Em vez de percorrer e desalocar objetos um a um, o CPU **recua o ponteiro de alocação da arena numa única instrução**. Toda a memória volátil evapora instantaneamente, e os recursos externos controlados por `LinearPtr` disparam os respetivos ganchos `@cleanup`.

**Porque importa:** a recuperação de erros passa a ter custo  $O(1)$  na arena, em vez de  $O(n)$  no número de objetos vivos — e sem qualquer risco de recurso órfão.

**Endpoints em pânico (cancelamento, não drop):** os endpoints de sessão não têm `Drop` (§2) — no unwinding, o runtime não os descarta em silêncio: envia ao par o **sinal de cancelamento**, recebido como um ramo normal do protocolo (em `Maybe~`, o rótulo `Closed` já é esse ramo). A recuperação mantém-se  $O(1)$  na memória da arena e  $O(\text{filhos})$  nas mensagens de cancelamento. (Prior art: *Exceptional Asynchronous Session Types* — Fowler, Lindley, Morris & Decova, POPL 2019.)

Listagem 7.1 – Pânico numa sub-arena: recuo  $O(1)$  e cleanup disparado

```
processBatch :: Arena %1 -> Channel Ack %1 -> IO ()
processBatch parent ack = withSubArena parent $ \sub -> do
  ctx <- initSSL                -- LinearPtr com @cleanup (§5)
  let buf = allocateCell sub 65536 -- vive na sub-arena
  when (corrupt buf) $ panic "lote inválido"
  -- — se 'panic' dispara, o Linear Unwinding executa, nesta ordem: —
  -- 1. o ponteiro da sub-arena recua numa instrução -> 'buf' evapora ( $O(1)$ ).
  -- 2. o @cleanup de 'ctx' corre: openssl_ctx_free(ctx).
  -- 3. 'ack' (endpoint, sem Drop) NÃO é descartado: envia Closed ao par (§7, §9).
  sendAck ack
```

## 08 Arquitetura do Compilador: Velocidade e Ergonomia (Salsa Engine)

Para um ciclo de *feedback* sub-milissegundo, o compilador rejeita a compilação em lote e assenta em quatro pilares:

### Separabilidade de Interfaces

Interface de assinaturas e implementação são rigidamente separadas. Alterar o corpo sem mudar a assinatura linear evita recompilar os módulos a jusante.

### Avaliação por Consultas

O compilador opera como base de dados reativa (Salsa). Muda um ficheiro → só o nó da AST afetado é invalidado.

### Typechecker Paralelo

A pureza nas fronteiras dos módulos deixa a inferência e a validação linear correr por todos os núcleos.

### Fast-Path Backend

Em `axion build --dev` desligam-se as otimizações agressivas, gerando binários de teste instantâneos.

### Ergonomia e Carga Cognitiva

A carga da Axióon é *topológica*: exige raciocinar sobre grafos — de posse, de comunicação e de acoplamento — que são implícitos. O princípio de mitigação preserva todas as garantias: **nenhum humano mantém na cabeça um grafo que o compilador já calcula**. A topologia é inferida, mostrada, explicada no erro e escondida até ser precisa — a semântica linear não muda.

- Níveis de capacidade (divulgação progressiva).** A Análise de Polaridade classifica cada módulo num nível e o compilador só expõe a maquinaria que o código usa — quem escreve L0 nunca vê linearidade nem topologia.
- Compilador Sócrático que mostra a topologia.** Sobre o Salsa (feedback sub-milissegundo), o LSP desenha o grafo inline — dono, ponto de morte (Auto-Drop), empréstimos, canal — convertendo simulação mental em consulta. Dois compromissos concretos desde a Fase 1: **códigos de erro estáveis** ( `axion --explain AX0042` ), emitidos em formato estruturado (JSON) para o LSP, ferramentas externas e assistentes de IA; e **sugestões *machine-applicable*** que o LSP aplica com um clique — foi isto, mais do que o texto dos erros, que decidiu a adoção do Rust. Exemplo completo na Listagem 8.3.
- Inferência que sugere, não que adivinha.** O compilador infere escopos `bound` , multiplicidades e pontos de `promote` , e no erro indica a correção exata. O humano escreve intenção; a topologia é derivada e verificada.
- Biblioteca de padrões topológicos provados.** Combinadores de alto nível ( `pipeline` / `parMap` em árvore, construtores seguros de grafos cíclicos sobre Tokens Fantasma, `withResource` ) encapsulam o raciocínio difícil uma vez.
- Menos conceitos + notação legível.** Tudo deriva de uma disciplina subestrutural única; aliases por palavras ( `shared` para `%0.5` , `peek` para `&?` , `coupled` para `~` ) deixam principiantes ler palavras e peritos usar símbolos.

| Nível | O que o programador vê                                                        | Garantias acumuladas |
|-------|-------------------------------------------------------------------------------|----------------------|
| L0    | Sintaxe de Haskell estrito puro; linearidade e regiões inferidas e invisíveis | Sem GC, sem fugas    |

|    |                                                                                                  |                                                  |
|----|--------------------------------------------------------------------------------------------------|--------------------------------------------------|
| L1 | Recursos lineares ( <code>%1</code> / <code>%0.5</code> ) e arenas                               | + posse e reset determinístico                   |
| L2 | Canais e session types ( <code>bound</code> )                                                    | + race- e deadlock-freedom                       |
| L3 | Acoplamento ( <code>~</code> / <code>Maybe~</code> ) e o tipo <code>Trit</code> (observe/Kleene) | + topologia de tipos; seleção branchless (geral) |

**Teto de nível por pragma (firewall de complexidade).** O nível é inferido pela Polaridade — e pode ser imposto: `{-# LEVEL L1 #-}` num módulo, ou `max-level` no manifesto do pacote (o pragma aperta o teto, nunca o alarga), transformam a divulgação progressiva de promessa cultural em **garantia mecânica**, à imagem do `#![forbid(unsafe_code)]` do Rust: «neste pacote ninguém escreve L3». O teto governa o que se *escreve*, não o que se *chama* — um módulo L1 pode depender de uma biblioteca L3; sem esta regra, a firewall estrangularia o ecossistema. Exemplo do erro na Listagem 8.4.

## Combinadores de padrões topológicos (solução 4)

Cada combinador prova a topologia difícil *uma vez* e expõe uma assinatura simples. O tipo `Bound s` marca as operações que têm de correr dentro de um nursery `bound`, e o parâmetro fantasma `s` impede que recursos escapem — o utilizador escreve a intenção, não o grafo.

Listagem 8.1 – Combinadores que encapsulam a topologia (assinaturas)

```
-- 'Bound s' is a computation that only runs inside a nursery 'bound arena $ do ...'.
-- The phantom parameter 's' prevents channels and resources from escaping the nursery.

-- withResource: acquires, uses and frees a linear resource at scope exit,
-- even on panic (Linear Unwinding). Hides the @cleanup threading.
withResource :: Acquire r %1          -- allocation + @cleanup bundled
              -> (r %1 -> IO a)       -- body; consumes 'r' exactly once
              -> IO a

-- parMap: applies 'f' in parallel; each worker is born as a CHILD of the nursery,
-- so the communication topology is a tree => deadlock-free by construction.
parMap :: (a %1 -> b %1)              -- linear per-element transformation
       -> Vector a %1                -- input; ownership moves to the workers
       -> Bound s (Vector b %1)      -- results, collected in order

-- stage: wraps a linear function into a pipeline stage.
stage :: (a %1 -> b %1) -> Stage a b %1

-- pipeline: links stages by linear channels in a chain (a path = a tree),
-- each stage on its own thread. Hides channel creation and teardown.
pipeline :: Stage a b %1
         -> Stage b c %1
         -> LinearStream a %1         -- source
         -> Bound s (LinearStream c %1) -- output

-- Usage: inside a nursery, with the Telescope operator (>|).
process :: Arena %1 -> LinearStream Packet %1 -> IO (LinearStream Result %1)
process arena src = bound arena $ do
  src |> stage decode
      |> stage validate
      |> stage apply
```

A outra metade da solução 4 é para **dados**: construir grafos cíclicos na heap sem `Rc` / `RefCell` nem coletor de ciclos. A chave (Tokens Fantasma, §3D) é separar os *handles* de topologia — `Cell s a`, não-lineares, logo várias arestas podem apontar para o mesmo nó — da permissão de mutação `Token s %1`, linear e única. O parâmetro fantasma `s` impede que uma `Cell` escape, pelo que o grafo inteiro se liberta num único reset de arena.

#### Listagem 8.2 – Grafos cíclicos seguros sobre Tokens Fantasma (assinaturas)

```
-- Cell s a : SHAREABLE handle to a node in region 's' (non-linear => allows cycles).
-- Region s : monad where the linear Token is threaded implicitly
--           (the first-class form of the 'using' block over Phantom Tokens).

-- withRegion: builds and reads, returning a PURE result. The 'forall s' (runST style)
-- prevents any Cell from escaping; the graph dies at the arena reset.
withRegion :: Arena %1 -> (forall s. Region s a) -> a

newCell :: a -> Region s (Cell s a)      -- allocates a node
get      :: Cell s a -> Region s a       -- reads the contents (borrow; no consume)
mutate   :: Cell s a -> (a -> a) -> Region s () -- rewrites under the linear permission

-- Usage: two nodes pointing at each other (a cycle), safely.
data Node s = Node { value :: Int, next :: Maybe (Cell s (Node s)) }

twoCycle :: Arena %1 -> Int
twoCycle arena = withRegion arena $ do
  a <- newCell (Node 1 Nothing)
  b <- newCell (Node 2 Nothing)
  mutate a (\n -> n { next = Just b }) -- a -> b
  mutate b (\n -> n { next = Just a }) -- b -> a (cycle!)
  va <- get a
  vb <- get b
  pure (value va + value vb)
```

**Efeito:** a carga cognitiva deixa de ser constante e mental — passa a ser paga só quando se usa o poder e amparada pela ferramenta. A «K2» não vira colina, mas deixa de se subir às cegas.

## Barreira de Entrada

Distinta da carga cognitiva (que pesa enquanto já se é fluente), a **barreira de entrada** é a primeira hora e a primeira semana. Cinco medidas baixam-na sem tocar nas garantias:

- 1 **L0 é o default e lê-se como Haskell estrito.** No dia 1 não se aprende nenhum conceito novo: escreve-se código puro estrito e a linearidade é inferida; os erros de linearidade só surgem ao optar por L1. Se sabes Haskell estrito, já lêes e escreves Axión — a sintaxe e o modelo mental transferem-se; o runtime por baixo é outro.
- 2 **Modo de ensino no compilador.** No primeiro erro de linearidade/escape, o compilador Socrático não só aponta — explica o conceito, mostra a correção mínima e liga à ideia. Aprende-se a disciplina através dos erros.
- 3 **Playground sem instalação.** O modo interpretado do compilador próprio (o fast-path de desenvolvimento), compilado para WASM, corre o primeiro programa no browser em segundos; o motor Salsa (sub-milissegundo) torna o ciclo escrever→ver vivo, sem fricção de toolchain.

- 4 **Caminho guiado L0→L3 («Axión by Example»).** Cada passo introduz exatamente um conceito novo, com exemplo executável — nunca se vê `~` / `bound` / trits antes de dominar o núcleo linear.
- 5 **Pontes de transferência + scaffolding.** Uma tabela «tu já sabes X, em Axión é Y» (os lifetimes do Rust são inferidos, `I0Ref ≈ Cell s`), e `axion new` gera um projeto que já compila — edita-se código vivo, não uma página em branco.

**Alvo:** a barreira desce de «O K2» para «O Muro, com vias fixas» — uma subida séria mas com percurso guiado, ao nível do Rust e não pior. O poder mantém um piso irredutível; a primeira experiência passa a ser suave.

## O compilador Socrático e a firewall em ação

As promessas acima em concreto — o texto exato que o programador vê. Cada diagnóstico tem o código estável, o porquê, a correção mínima (aplicável pelo LSP com um clique) e a porta para a explicação longa.

**Gamas de códigos de diagnóstico.** Os códigos `AXnnnn` são estáveis e agrupados por família: `AX00xx` linearidade e posse (uso-após-consumo, *must-use*, escape); `AX01xx` níveis e *firewall* de capacidade; `AX02xx` regiões e arenas; `AX03xx` canais e session types; `AX04xx` FFI e fronteira de confiança. O registo nasce na Fase 1 (§17) e cada código tem `--explain` + saída JSON.

Listagem 8.3 – Um diagnóstico Socrático completo (AX0001)

```
error[AX0001]: uso após consumo de um valor linear
--> src/Crypto.axi:42:24
|
|
40 |     let key = deriveKey buf
|           --- a posse de 'buf' moveu-se aqui
|
|
42 |     let sum = checksum buf
|           ^^^ 'buf' já tinha sido consumido
|
|
= porquê: buf : Buffer U8 %1 — posse plena, exatamente um dono (§2)
= ajuda [machine-applicable — o LSP aplica com um clique]:
  usa a variante que devolve a posse:
    40 |     let (key, buf') = deriveKeyKeep buf
    42 |     let sum = checksum buf'
= leitura partilhada? split para %0.5 (§2)
= explicação completa: axion --explain AX0001
```

Listagem 8.4 – A firewall de nível a recusar L2 num módulo L1

```
error[AX0101]: construção de nível L2 num módulo com teto L1
--> src/Pipeline.axi:14:5
|
|
14 |     bound arena $ do
|       ^^^^^ 'bound' (canais + session types) é uma construção L2
|
|
= teto do módulo: L1  ({-# LEVEL L1 #-}, linha 1)
= teto do pacote: L2  (axion.toml: max-level = "L2")
= duas saídas:
  1. sobe o teto do módulo para {-# LEVEL L2 #-}; ou
  2. move a concorrência para um módulo L2 e expõe daqui uma função pura.
```

**Nota de honestidade técnica:** não há computação quântica real. «Quântico» é aqui uma metáfora para uma topologia de tipos derivada da lógica linear. O valor prático é representar escolhas exclusivas e leituras não-destrutivas com custo zero. Em concreto, `~` e `Maybe~` não transportam efeito não-local instantâneo (não há *no-signaling* quântico): são açúcar sintático sobre os canais lineares da §06 — todo o «impacto» remoto é uma mensagem entregue no próximo `observe` (ver §09.D). Por isso o mecanismo chama-se **acoplamento por sessão** (alias `coupled`), e não «emaranhamento»: o nome descreve o que é — dois *endpoints* duais de fim de vida ligado — sem prometer a não-localidade que a própria semântica nega. «Emaranhamento» fica só como intuição informal, nunca como termo normativo.

## A. Conjunção Aditiva ( `A & B` ) e o Operador Telescópio ( `|>` )

O tipo `A & B` oferece **dois potenciais alternativos**: uma escolha adiada entre eles, sem `.clone()`. Representação: quando ambos os potenciais são projeções dos mesmos dados base (o caso comum), partilham o mesmo espaço físico na Stack, sem tags de enumeração — custo zero; no caso geral, `&` compila para um fecho defuncionalizado (§3) que produz o potencial escolhido — custo de um fecho, nunca de uma cópia. A introdução de um valor `A & B` faz-se com `superpose a b`; a eliminação é o `case` (colapso) ou o `&?` (projeção). Na lógica linear, `&` significa «podes escolher um dos dois, mas não ambos». O **Operador Telescópio** `|>` transporta a massa do recurso de forma linear e contínua ao longo das cadeias de transformação.

**Intuição (analogia, não definição).** Podes pensar num `A & B` como uma *superposição*: dois potenciais que coexistem até escolheres, e o `case` é o *colapso* que fixa um e poda o outro. É uma boa muleta de memória — mas a analogia para aqui: a escolha é **determinística** (escolhe o consumidor, não o acaso), não há probabilidades nem física de partículas. Formalmente, é apenas a **conjunção aditiva** da lógica linear (§16).

Listagem 9.1 – `superpose` introduz `A & B`; `&?` espreita, `case` colapsa

```
-- 'superpose a b' builds an (A & B): two potentials, NOT a copy, NOT a channel.
-- Common case: both are projections of the same base data => same physical
-- space on the Stack, no enum tag => zero cost (§9.A).
plan :: Config %1 -> (Report & Report)
plan cfg = superpose (draftReport cfg) (finalReport cfg)

usePlan :: (Report & Report) %1 -> IO ()
usePlan p = do
  -- &? peeks ONE potential without collapsing (read-only borrow, §2) – both survive.
  let preview = p &? Draft
  printSummary preview
  -- 'case' collapses destructively: picking a branch PRUNES the other from the flow.
  case p of
    Final r -> publish r          -- 'Draft' is now unreachable; touching it is AX0001
```

## B. Colapso e Projeção (pattern matching linear)

O bloco `case` realiza o **colapso destrutivo**: ao escolher um dos dois potenciais, o ramo oposto é podado do grafo de fluxo, impedindo o seu uso posterior. Para leituras não-destrutivas usa-se a **Projeção Persistente** `&?`, que espreita um potencial sem forçar o colapso. Formalmente, `&?` não é uma operação de canal: é um



*borrow* apenas-leitura (a mesma elisão de empréstimos da §02), cujo tempo de vida não excede o do recurso e não transfere posse — por isso preserva a linearidade de `&`.



Fig. 9.1 — Peek (&?) preserva ambos os ramos; case colapsa e poda o oposto.

### C. Heurísticas de Escopo para Paradigmas Híbridos

| Bloco      | Responsabilidade única                                                                                                                                                              |
|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| imperative | Algoritmos puramente matemáticos e buffers de baixo nível (sem IO).                                                                                                                 |
| using      | Grafos dinâmicos sob um único Token de mutação numa arena.                                                                                                                          |
| bound      | Nursery de concorrência estruturada: os canais nascem confinados ao nursery e só se ligam em topologias acíclicas, o que exclui a espera cíclica — deadlock-freedom por construção. |

**Um conceito, quatro superfícies.** Os quatro blocos ( `imperative` , `using` , `bound` , `susp` ) desugaram para uma única construção subjacente: um escopo que concede uma **capacidade linear** — buffer mutável, Token de mutação, nursery, suspensão — com o parâmetro fantasma `s` a impedir que a capacidade escape. O typechecker tem uma regra com quatro instâncias, e o conceito ensina-se uma vez; as palavras-chave distintas mantêm-se à superfície porque dizem ao leitor a responsabilidade do bloco sem olhar para os tipos (tabela acima). Desugaring na Listagem 9.2.

```
Listagem 9.2 – Desugaring normativo dos quatro blocos para scope[C]

-- Todos os blocos com escopo reduzem à MESMA construção do Axióm Core:
--   scope[C] s. e   -- concede a capacidade linear C; o fantasma 's' impede o escape.

imperative $ do ... ==> scope[MutBuffer s] s. ...  -- mutação local (§4)
using tok   $ do ... ==> scope[Token s]    s. ...  -- encadeia o token (§3D)
bound arena $ do ... ==> scope[Nursery s]   s. ...  -- arena + scheduler (§11)
susp       $ do ... ==> scope[Suspend s]   s. ...  -- constrói um Susp (§3A)

-- Uma regra de tipagem, quatro instâncias: o typechecker verifica 'scope'
-- uma vez; cada palavra-chave só escolhe a capacidade e o seu açúcar.
```

### Listagem 9.3 – String em A & B e validador acoplado confinado

```
-- Additive conjunction (&): the same physical space holds two logical potentials.
data Term s      = Term (String & String) -- Original & Modified
data Validator s = Validator (Ptr HardwareScreen)

processTerm :: Arena %1 -> IO ()
processTerm arena = bound arena $ do
  -- Coupled pair (~): dual session endpoints, linked lifetimes.
  (term ~? validator) <- makeCoupledPair arena "run" "running"

  -- Persistent Projection (&?): peeks WITHOUT collapsing the choice.
  let baseText = term &? Original

  -- 'case' collapses destructively; the opposite branch is pruned from the flow.
  case term of
    Modified "running" -> checkImpact validator

-- 'validator' is the endpoint coupled with 'term' (the Validator side).
checkImpact :: (Validator s %1 Maybe~ Term s %1) -> IO ()
checkImpact validator =
  case observe validator of
    Live v -> renderText v "Term in base state."
    Closed v0ld -> adjustLayout v0ld "running" -- peer closed (cancel received)
    Pending v -> deferRender v -- message not arrived yet (poll)
```

## D. ~ e Maybe~ como Canais Lineares (a semântica do «acoplamento»)

O «acoplamento» entre *threads* não é um efeito físico nem instantâneo: é **açúcar sintático** que o compilador reduz, por *desugaring* normativo, aos **endpoints lineares** da §06. É essa redução que torna o efeito «não-local» *mediado por um canal linear* — e, por isso, seguro.

| Superfície (açúcar)             | Semântica operacional (§06)                                                                |
|---------------------------------|--------------------------------------------------------------------------------------------|
| $A \sim B$                      | Par de endpoints duais: $(\text{Ep } \%1\ S) \otimes (\text{Ep } \%1\ (\text{dual } S))$ . |
| $A\ \text{Maybe}\sim B$         | Canal com escolha de três rótulos, um por Trit: $\text{Ep } \%1\ (\text{Offer}\dots)$ .    |
| <code>makeCoupledPair</code>    | <code>newChannel</code> — cria e devolve os dois endpoints duais.                          |
| <code>case ... (colapso)</code> | <code>send</code> no endpoint local (enfileira a mensagem).                                |
| <code>observe</code>            | <code>recv</code> / <code>branch</code> não-bloqueante — devolve o Trit.                   |
| <code>&amp;?</code>             | <i>Borrow</i> apenas-leitura local (§02) — não usa canal.                                  |

### Listagem 9.4 – Desugaring normativo de ~ / Maybe~ para endpoints lineares

```
-- '~' links the lifetimes through a pair of dual endpoints.
A ~ B      ==> (Ep %1 S) \otimes (Ep %1 (dual S))

-- 'Maybe~' is a channel whose branch has exactly the 3 Trit labels (ternary engine).
A Maybe~ B ==> Ep %1 (Offer { Live : S1 -- +1 TPlus
                             , Closed   : S2 -- -1 TMinus
                             , Pending  : S3 }) -- 0 TZero (not arrived)
```

**Garantia (teorema):** «zero data races» é corolário da **Fidelidade de Sessão** — cada endpoint pertence a exatamente uma thread (linearidade `%1`), logo nunca existem dois acessos concorrentes ao mesmo endereço. O «impacto não-local» não é instantâneo: é uma mensagem entregue quando o par executa `observe` — causal e ordenada pelo canal.

**Deadlocks:** a Fidelidade de Sessão garante ausência de data races; a ausência de deadlocks vem do bloco `bound`, que é um *nursery* de concorrência estruturada: os canais nascem por spawn dentro do nursery e os combinadores da §8 só os ligam em configurações que preservam a aciclicidade — o invariante é a **topologia de comunicação acíclica** (uma floresta): as cadeias do `pipeline` entre irmãos são caminhos, logo acíclicas, e a espera cíclica torna-se impossível de exprimir. Deadlock-freedom é um corolário da tipagem, não uma análise à parte. (Isto é ortogonal aos dados cíclicos da §3D: o grafo de *dados* na heap pode ter ciclos; o grafo de *comunicação* entre threads não.)

## Metateoria e verificação (custo zero)

As garantias acima (preservação, progresso, fidelidade de sessão e deadlock-freedom) são fundadas em lógica linear (*Propositions as Sessions*): a deadlock-freedom e o progresso são corolários da eliminação de cortes — a *proof net* acíclica é exatamente a árvore que o `bound` impõe. A prova estabelece-se por uma escada estática: testes por propriedades, model-checking dos protocolos (CFSMs), metateoria mecanizada (Iris/Actris) e, por fim, um verificador de referência extraído, provado fiel à semântica.

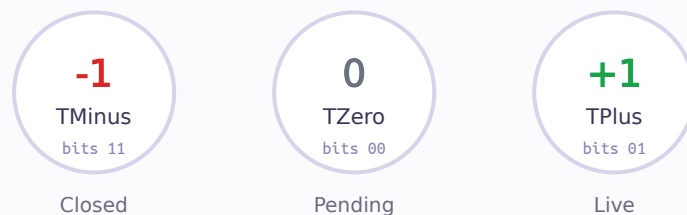
**Sem monitores em runtime:** a rede de segurança contra um erro de metateoria não vigia o programa em execução — vigia o compilador. É, por isso, um **verificador de referência em tempo de compilação** (*translation validation*) que faz cross-check das decisões do typechecker de produção. Custo de runtime: **zero** — a latência zero mantém-se de ponta a ponta. Um check dinâmico só pode surgir, opcional e apagável em `--release`, na fronteira `foreign` / `trusted`, onde já se aceita confiança.

## 10 O Tipo `Trit` e a Compactação Ternária (Opcional)

O `Trit` é um **enum de três estados** — `-1` (TMinus), `0` (TZero), `+1` (TPlus) — que dá corpo aos desfechos de `observe` (§9) e ao terceiro valor da **Lógica de Kleene** no typechecker (o `0` como «desconhecido/pendente»). **Não é um «motor» nem uma inovação de hardware**: é um tipo ordinário, o caso  $N=3$  do mecanismo geral de *sum types* — à imagem do `Ordering` (Less/Equal/Greater) do Rust. Conserva o nome próprio porque tem operações próprias (negação por troca de sinal, os conectivos de Kleene). Duas propriedades merecem detalhe: a **seleção *branchless*** — que é partilhada por *todos* os enums pequenos, não é específica do ternário — e a **compactação de alta densidade**, uma otimização **opcional de biblioteca**, não uma camada do núcleo.

### A. Representação e Execução *Branchless*

Em hardware clássico, um `Trit` ocupa 2 bits de um registo (como qualquer enum de 3–4 variantes). Um `case` sobre o `Trit` compila sem saltos **quando os ramos selecionam valores** (cmov/máscaras); ramos que executam código distinto continuam a ser saltos reais — o ganho *branchless* aplica-se à seleção de dados, não ao fluxo de controlo arbitrário. **Nota importante**: esta seleção sem saltos **não é propriedade do ternário** — é do backend de enums pequenos (é mais forte para  $N=2-4$ , acima disso vira *jump table*). O `observe` devolve um `Trit` e beneficia dela como qualquer outro enum; o ternário não acrescenta nada aqui além da simetria  $-1/0/+1$  conveniente para Kleene.



3 estados simetricos -> 'observe' vira aritmetica de mascaras (selecao de valores sem saltos).

Fig. 10.1 — Os três trits balanceados e a sua codificação em 2 bits.

Listagem 10.1 — `observe` devolve um `Trit`; seleção de valores por máscaras

```
-- 'observe' returns a Trit; a value-selecting 'case' compiles to mask arithmetic.
checkLink :: Endpoint s %1 -> Trit
checkLink ep = observe ep
-- TPlus (+1 / bits 01) -> Live : link intact
-- TMinus (-1 / bits 11) -> Closed      : remote resource destroyed
-- TZero (0 / bits 00) -> Pending      : asynchronous indeterminacy
```

### B. Trit-Packing de Alta Densidade (Compressão Base-243) — biblioteca opcional `TritVec`

Esta é a **única** parte genuinamente específica do ternário — e vive numa **biblioteca opcional** (`TritVec`), não no núcleo da linguagem. Alocar 2 bits para 3 estados desperdiça 25%; agrupar **cinco trits num único byte** ( $3^5 = 243 \leq 256$ ) eleva a ocupação a **95%** (densidade informacional 99,1% vs 79,2% no formato solto de 2 bits). O **trade-off, honesto**: o packing a 2 bits/trit (75%) é trivial — só *shifts*, sem conversão de radix; o base-243 (99%) exige conversão de radix no *unpack*, logo **só compensa quando se está genuinamente limitado por largura de banda de memória**. O caso de uso onde paga de facto é concreto e atual: **arrays massivos de 3 estados** — em particular **ML quantizado ternário** (redes de pesos  $\{-1, 0, +1\}$ , ~1,58 bits/peso), onde  $5\times$  menos *cache misses* engolem o custo do *unpack*. Fora daí, a densidade não justifica a complexidade — daí ser biblioteca, e não obrigatória.

Trit-Packing Base-243: 5 trits -> 1 byte ( $3^5 = 243 \leq 256$ )



1 Byte compactado — ocupação 95% (vs 75% no formato 2 bits solto)

Na RAM: guardado compactado. Nos registos (RAX/SIMD): unboxing transparente para 2 bits branchless via LUT de 256 entradas + PDEP (x86\_64) ou tabela + aritmética (ARM).

Fig. 10.2 — Cinco trits posicionais compactados num byte, expandidos sob demanda.

## C. Fusão de Instruções e Análise de Polaridade

**Aceleração por silício (na biblioteca):** o desempacotamento base-243 é uma **conversão de radix** (div/mod por potências de 3) — não uma extração de bits — e o codec ideal depende do alvo: LUT de 256 entradas → 10 bits, com `PDEP (BMI2)` a espalhar os pares de bits em x86\_64, ou *table lookups* SIMD ( `PSHUFB` / `TBL` ) para desempacotar vetores largos de uma vez. Nota de microarquitetura: PEXT/PDEP são rápidos em Intel e AMD Zen 3+, mas **microcodificados (lentos) em Zen 1/2** — daí a biblioteca não depender deles: seleciona por alvo um codec uniformemente rápido (LUT/SIMD).

**Análise de Polaridade Estática:** o `Trit` enquanto tipo está sempre disponível (é um enum); o que a Polaridade decide é apenas **onde vale a pena usar o formato compactado**. Se um valor é puramente linear e determinado estaticamente, nunca se paga *packing* — trabalha-se com o enum solto, Assembly clássico. A compactação é a exceção medida, não a regra.

## 11 Pipeline de Compilação Consolidado (Axión Core)

O pipeline é uma cadeia própria, de raiz, em **cinco estágios** — a arquitetura conceptual (Core intermédio, passes de otimização, backend nativo) segue a escola do GHC, mas nenhum estágio reutiliza o seu código ou runtime. O **Parser** expande os blocos com escopo ( `using` / `bound` / `imperative` / `susp` ) e a sintaxe de Trits. O **Typechecker** (paralelo) corre a linearidade, a Análise de Polaridade e a Lógica de Kleene. A baixada para o **Axión Core** — um IR estrito e linear, sem *thunks* — injeta o Auto-Drop, os ganchos `@cleanup` e os resets de arena. O **Otimizador** aplica Stream Fusion e defuncionalização (a compactação Base-243 é um passe *opcional*, só quando se usa `TritVec`, §10). O **Backend nativo** emite Assembly com seleção branchless de enums pequenos (BMI2/ARM): fast-path interpretado/Cranelfit em `--dev`, LLVM em `--release`. As garantias vivem no Typechecker; os estágios seguintes só as materializam.

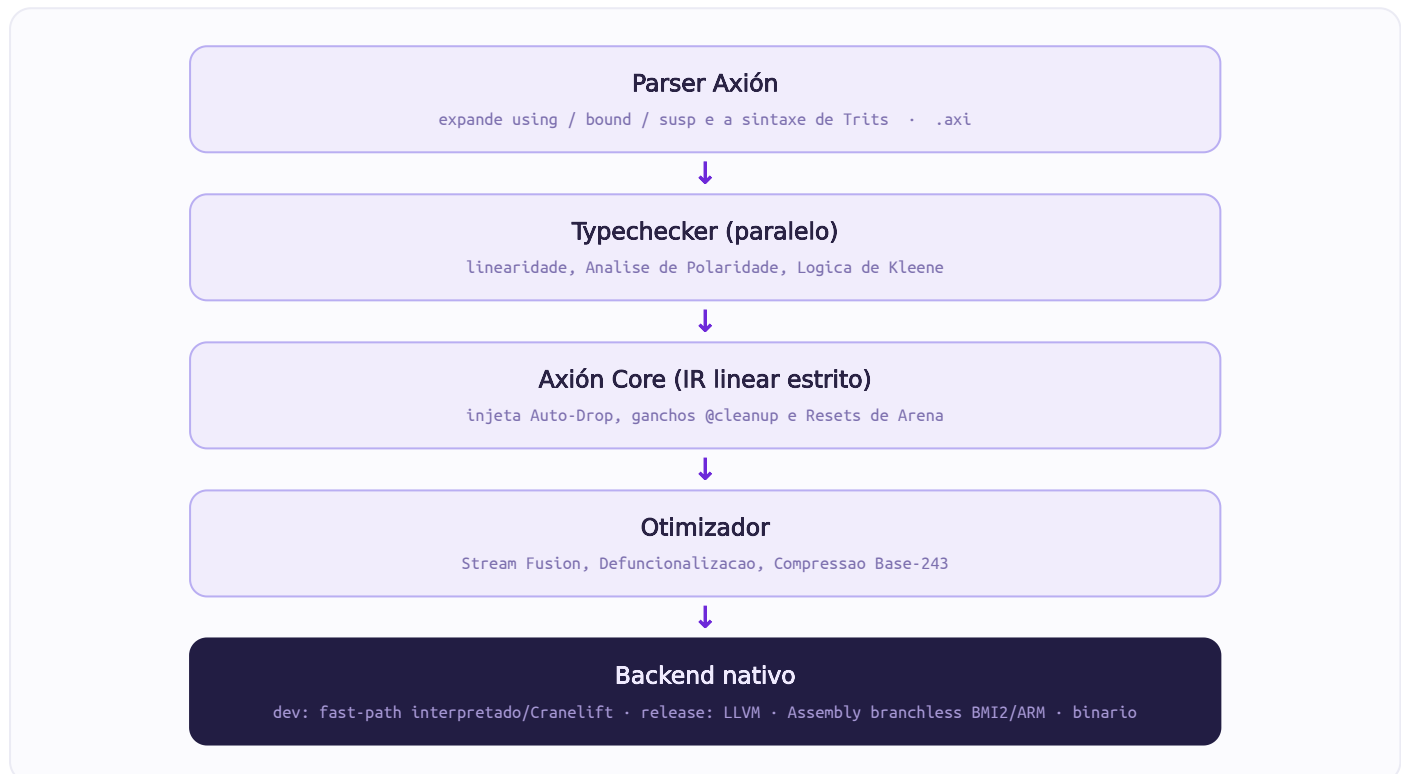


Fig. 11.1 — Do código-fonte ao binário branchless, em cinco estágios — num compilador próprio.

### Runtime da Axión (execução e concorrência)

Sem o GC não há stacks móveis nem green threads herdadas — o runtime é definido por cinco decisões:

- Scheduler M:N com work-stealing**: N threads do SO (uma por núcleo), com `io_uring/epoll` integrado para IO assíncrono.
- Tarefas = continuações defuncionalizadas** (a mesma maquinaria do `Susp`, §3), alocadas na arena do nursery: **uma nursery é uma arena com scheduler**. Spawn é bump-pointer; o fim do `bound` liberta tudo num reset. Os pontos de suspensão são exatamente as operações de canal — visíveis nos tipos ( `Bound s`, endpoints `%1` ) — pelo que não há `async/await` na superfície nem *function coloring*; `imperative` nunca suspende (§9.C), logo computação pesada corre direta no worker.
- Stacks**: recursão entre pontos de suspensão usa a stack do worker (com guard pages); recursão através de suspensões aloca os frames CPS na arena do nursery — sem stacks segmentadas nem cópia de stacks.
- Pânico e cancelamento**: matar uma tarefa = sinal de cancelamento aos endpoints filhos (o ramo implícito do protocolo, §7) + reset da sua fatia de arena —  $O(1)$  na memória,  $O(\text{filhos})$  nas mensagens.

- 5 **FFI bloqueante** corre num pool dedicado de threads do SO; callbacks C que re-entram na Axióon usam a stack de quem chama — fronteira documentada junto do `trusted`.

**Notas de honestidade:** `parMap` cria `min(n, núcleos×k)` tarefas com *chunking* — não literalmente uma por elemento. É um dividendo do desenho: com escalonamento determinístico opcional ( `--deterministic` ), gravar a ordem dos `observe` chega para *record/replay* exato de execuções concorrentes.





## Gramática EBNF (normalizada)

```
(* Axion Language Grammar Specification - Version 0.2 *)

(* Core Type System and Constraints *)
AxionType      ::= PrimitiveType | CompositeType | LinearType | FractionalType
                | AdditiveType | CoupledType | ConditionalCoupled
                | TernaryPrimitive ;
LinearType     ::= AxionType "%1" ;
FractionalType ::= AxionType "%0.5" ;      (* shared-read permission *)
AdditiveType   ::= AxionType "&" AxionType ; (* additive conjunction: choose one *)
CoupledType    ::= AxionType "~" AxionType ;
ConditionalCoupled ::= AxionType "Maybe~" AxionType ;
TernaryPrimitive  ::= "Trit" ;

(* Block Structures and Contextual Scopes *)
AxionExpression ::= ScopedBlock | ObserveExpression | ForceExpr
                  | SuperposeExpr | PersistProjection
                  | TrustBlock | AssumeExpr | StandardExpression ;
ScopedBlock     ::= ScopeHead BlockArg ;
ScopeHead       ::= "using" Identifier | "bound" Identifier
                  | "imperative" | "susp" ;
BlockArg        ::= "$"? DoBlock ;          (* BlockArguments: '$' optional *)
DoBlock         ::= "do" Statement { Statement } ;
ObserveExpression ::= "observe" Expression ;
PersistProjection ::= Expression "&?" Identifier ;

(* Prefix Forms and Introduction *)
ForceExpr       ::= "force" Expression ;      (* forces a Susp *)
SuperposeExpr   ::= "superpose" Expression Expression ; (* builds A & B *)

(* Patterns and Binding *)
Pattern         ::= Identifier | CoupledPattern | WildcardPattern ;
CoupledPattern  ::= "(" Pattern "~?" Pattern ")" ; (* dual coupled pair *)
BindStatement   ::= Pattern "<-" Expression ;

(* Foreign Function Interface and Lifecycles *)
FFIDeclaration  ::= "foreign" "import" "ccall" StringLiteral
                  IsolationAttribute? CleanupAttribute? Identifier ":" AxionType ;
CleanupAttribute ::= "@cleanup" ;      (* binds to the next declaration *)
IsolationAttribute ::= "@trusted"      (* default se omitido: audita, não isola *)
                  | "@isolate" "(" IsoTarget ")" (* páginas protegidas por hardware *)
                  | "@sandbox" "(" IsoTarget ")" ; (* SFI in-process *)
IsoTarget        ::= "mpk" | "wasm" ;

(* Operators *)
PipeOperator    ::= "|>" ;

(* Trust boundary – there is NO 'unsafe'; raw access is a visible effect *)
TrustBlock      ::= "trusted" BlockArg ;      (* audited TCB region *)
AssumeExpr      ::= "assume" Expression StringLiteral "$" Expression ;
                (* discharge one fact + mandatory justification *)
RawEffect       ::= "Raw" | "Foreign" ;      (* capability effect in a return type *)

(* Capability-ceiling pragma (level firewall, §8) – top of module *)
LevelPragma     ::= "{-# "LEVEL" ( "L0" | "L1" | "L2" | "L3" ) "#-}" ;
```

**Cobertura da gramática:** os blocos com escopo ( `using` , `bound` , `imperative` , `susp` ) são aplicação sobre um `do-block` ( `$ do` , com `$` opcional via *BlockArguments*). `force` e `superpose` têm produções próprias, e `~?` é um *padrão* de desestruturação do par dual (não uma expressão). As primitivas de arena ( `promote` , `arena_mark` , `arena_release` , `split` / `join` ) são funções normais, sem sintaxe dedicada. `trusted` é um bloco e `assume` uma expressão prefixa (fronteira de confiança, §5); `Raw` / `Foreign` são efeitos que aparecem no tipo de retorno.

**Base e layout (normativo).** A EBNF acima especifica apenas as *deltas* da Axión. Tudo o resto — declarações `data` , `let` / `where` , `case ... of` , assinaturas de tipo, módulos e `import` — segue **Haskell 2010**, incluindo a **regra de layout** (indentação significativa). O `{-# LEVEL #-}` é um pragma de topo de módulo ( `LevelPragma` ).

**Desambiguação de `case`.** A mesma sintaxe `case e of ...` tem três semânticas, escolhidas pelo **tipo de `e`** , sem ambiguidade sintática: um ADT normal → *pattern match*; um `A & B` → colapso da escolha aditiva (§9.B); um `Trit` → ramo ternário (§10). O typechecker resolve pela tipagem do escrutinado.

**Frações (V0.2).** As multiplicidades fracionárias limitam-se a **metades**: `split` divide `%1` em dois `%0.5` e `join` recombina-os. A subdivisão adicional ( `%0.25` , frações de Boyland gerais) fica *fora* do âmbito da V0.2 — um `%0.5` é leitura partilhada indivisível.

## Núcleo herdado de Haskell 2010 (em vigor, não repetido acima)

Para um implementador, o subconjunto de Haskell 2010 que a Axión adota **sem alterações sintáticas** — só a semântica muda (estrita, sem GC):

- **Declarações:** `data` / `newtype` / `type` , classes e instâncias, assinaturas `::` , definições de função com *guards* e `where` .
- **Expressões:** `let` / `in` , `case ... of` , `if/then/else` , lambdas `\x ->` , aplicação e secções de operador, `do` -notation.
- **Padrões:** construtores, literais, *wildcards* `_` , *as-patterns* `@` — mais o `~?` da Axión (acima).
- **Módulos:** `module ... where` , `import` (qualificado/oculto), listas de exportação.
- **Léxico e layout:** literais numéricos/string/char, comentários `--` / `{- -}` , e a **regra de layout** por indentação.

**Diferenças semânticas (não sintáticas):** avaliação **estrita** por omissão (preguiça só via `Susp` , §3A); sem GC; a linearidade e as regiões são inferidas e verificadas por cima desta sintaxe. Assim, quem lê Haskell lê Axión — o que muda está por baixo.

## Notação de Referência

Os símbolos densos da Axión derivam todos de uma disciplina subestrutural única. Cada um tem um *alias* por palavra, para principiantes lerem palavras e peritos usarem símbolos.

| Símbolo             | Alias                | Significado                                                                                  |
|---------------------|----------------------|----------------------------------------------------------------------------------------------|
| <code>%1</code>     | <code>linear</code>  | Posse plena; consumido exatamente uma vez — pelo programador ou pelo <code>Drop</code> (§2). |
| <code>%0.5</code>   | <code>shared</code>  | Permissão fracionária; leitura partilhada ( <code>split</code> / <code>join</code> ).        |
| <code>&amp;</code>  | <code>with</code>    | Conjunção aditiva: dois potenciais, escolher um.                                             |
| <code>&amp;?</code> | <code>peek</code>    | Projeção persistente: espreita sem colapsar ( <i>borrow</i> ).                               |
| <code>~</code>      | <code>coupled</code> | Par de <i>endpoints</i> lineares duais (açúcar de canal).                                    |

|                     |              |                                                                                                        |
|---------------------|--------------|--------------------------------------------------------------------------------------------------------|
| Maybe~              | maybeCoupled | Canal com <i>branch</i> tri-estado (um por Trit).                                                      |
| ~?                  | —            | Padrão que desestrutura o par acoplado.                                                                |
| >                   | pipe         | Operador Telescópio: encadeia transformações lineares.                                                 |
| ⊗                   | —            | Tensor (lógica linear): dois recursos independentes.                                                   |
| Trit                | —            | Ternário balanceado: −1 / 0 / +1 (TMinus/TZero/TPlus).                                                 |
| @cleanup            | —            | Gancho de limpeza automática de um <code>LinearPtr</code> (FFI).                                       |
| @isolate / @sandbox | —            | Isolamento opt-in do <code>foreign</code> : <code>mpk</code> (páginas) / <code>wasm</code> (SFI), \$5. |
| trusted             | assume       | Fronteira de confiança auditada (TCB); não há <code>unsafe</code> .                                    |

# 13 Grandes Quadros Comparativos de Engenharia

Posicionamento honesto face aos concorrentes diretos — incluindo o **OCaml com modes**, o mais próximo da tese da Axióon (§16). Legenda: 🏆 liderança (garantido hoje) · 🟢 bom · 🟡 médio · 🔴 fraco · — n/a · +🎯 = a marca à esquerda é a **posição teórica de desenho**, ainda por provar/medir.

## A. Segurança (Safety)

|                                              | Linguagem C | Rust                             | Haskell Original                                    | OCaml (modes)                                  | Axióon V0.2                                              |
|----------------------------------------------|-------------|----------------------------------|-----------------------------------------------------|------------------------------------------------|----------------------------------------------------------|
| Uso-Após-Livre / Double-Free                 | 🔴 Unsafe    | 🏆 Zero                           | 🏆 Zero                                              | 🏆 Zero<br>GC                                   | 🏆 Zero<br>Via Auto-Drop                                  |
| Recurso esquecido (handle, socket, endpoint) | 🔴 Nenhuma   | 🟡 Lint<br>#[must_use] é só aviso | 🔴 Nenhuma                                           | 🟡 Parcial<br>modes: uniqueness; must-use fraco | 🏆 Erro de compilação<br>Sem Drop ⇒ must-use (§2, AX0002) |
| Proteção contra Data Races                   | 🔴 Nenhuma   | 🏆 Absoluta                       | 🟡 Parcial<br>Puro sim; IOREf/MVar sem sincronização | 🟡 Parcial<br>multicore 5; sem session types    | 🏆 Absoluta<br>Canais Lineares                            |
| Proteção contra Deadlocks                    | 🔴 Nenhuma   | 🔴 Nenhuma<br>Não previne         | 🟡 Parcial<br>Deteta em runtime                      | 🔴 Nenhuma                                      | 🏆🎯 Por construção<br>bound (§9); metateoria a mecanizar  |
| Exposição a blocos unsafe                    | 🔴 Constante | 🟡 Média                          | 🟢 Mínima<br>Só no FFI                               | 🟢 Mínima<br>Obj.magic / FFI                    | 🟢 Mínima<br>Sem unsafe; trusted/foreign isolado          |
| Fugas de RAM / Space Leaks                   | 🔴 Alta      | 🟢 Mínima                         | 🔴 Crítica<br>Thunks                                 | 🟢 Mínima<br>estrito + GC                       | 🏆 Zero<br>Strict + NLL Arenas                            |
| Fugas de RAM em Pânicos                      | 🔴 Alta      | 🟢 Mínima                         | 🟢 Mínima                                            | 🟢 Mínima<br>GC; Fun.protect                    | 🏆 Zero<br>Linear Unwinding                               |

## B. Performance (Velocidade Bruta e RAM)

|                               | Linguagem C | Rust                           | Haskell Original    | OCaml (modes)                         | Axióñ V0.2                                   |
|-------------------------------|-------------|--------------------------------|---------------------|---------------------------------------|----------------------------------------------|
| Latência de Execução (Jitter) | 🏆 Zero      | 🏆 Zero                         | 🔴 Alta<br>Pausas GC | 🟡 Baixo<br>GC incremental (5)         | 🏆🎯 Zero<br>Sem GC — por medir                |
| Cálculo Numérico / Álgebra    | 🏆 Máxima    | 🏆 Máxima                       | 🔴 Baixa             | 🟡 Boa<br>floats boxed; Flambda        | 🏆🎯 Máxima<br>SIMD imperativo — por medir     |
| Alocações / Cópias Ocultas    | 🟢 Poucas    | 🟢 Poucas<br>Explícitas, como C | 🔴 Muitas            | 🟡 Muitas<br>minor heap; modes reduzem | 🏆🎯 Ótimas<br>Destructive Updates — por medir |
| Indireção em Closures         | 🏆 Zero      | 🟢 Baixa                        | 🔴 Alta              | 🟡 Média<br>closures alocam            | 🏆🎯 Zero<br>Unboxing de Susp — por medir      |
| Consumo de Memória RAM        | 🏆 Mínimo    | 🏆 Mínimo                       | 🔴 Alto              | 🟡 Moderado<br>overhead do GC          | 🏆🎯 Mínimo<br>NLL Arenas — por medir          |

## C. Velocidade do Compilador e Feedback Loops

|                          | Linguagem C                          | Rust                                    | Haskell Original          | OCaml (modes)                | Axióñ V0.2                                              |
|--------------------------|--------------------------------------|-----------------------------------------|---------------------------|------------------------------|---------------------------------------------------------|
| Compilação Fria          | 🏆 O mais rápido<br>Tradução direta   | 🔴 O mais lento<br>Monomorfização pesada | 🟢 Moderado<br>Core do GHC | 🏆 Rápido<br>famosamente      | 🟢🎯 Rápido / Modular<br>Typechecker paralelo — por medir |
| Recompilação Incremental | 🏆 Instantâneo<br>Relink de .o        | 🔴 Lento<br>Fronteiras grossas de crates | 🟢 Rápido<br>Por módulo    | 🟢 Rápido<br>por módulo, dune | 🏆🎯 Instantâneo<br>Salsa — por medir                     |
| Diagnósticos LSP         | 🟡 Lento<br>Reparse de headers        | 🟡 Moderado<br>Carga do rust-analyzer    | 🟢 Rápido<br>Bytecode GHCi | 🏆 Rápido<br>merlin           | 🏆🎯 Sub-ms<br>Separabilidade — por medir                 |
| Custo das Otimizações    | 🟢 Baixo / Moderado                   | 🔴 Extremo<br>Pipeline denso do LLVM     | 🟢 Moderado<br>Core padrão | 🟢 Moderado<br>Flambda        | 🏆🎯 Zero em Dev<br>Fast-Path Toggle — por medir          |
| Ciclo Edit-and-Run       | 🏆 Instantâneo<br>Compila+linca+corre | 🔴 Doloroso<br>cargo run trava no linker | 🟢 Rápido<br>via REPL/ghci | 🟢 Rápido<br>utop             | 🏆🎯 Instantâneo<br>axion run --dev — por medir           |

## D. Ergonomia do Programador e Carga Cognitiva

|                                | Linguagem C                       | Rust                                | Haskell Original             | OCaml (modes)                    | Axión V0.2                                |
|--------------------------------|-----------------------------------|-------------------------------------|------------------------------|----------------------------------|-------------------------------------------|
| Verbosidade (Boilerplate)      | 🔴 Excessiva<br>Headers, macros    | 🟡 Alta<br>Lifetimes explícitos      | 🏆 Mínima                     | 🟢 Baixa<br>conciso; modes anotam | 🟢 Baixa<br>Sem manual drop                |
| Rastreo Mental de Memória      | 🔴 Crítico<br>malloc/free manual   | 🔴 Alto<br>Luta com o borrow checker | 🏆 Zero<br>O GC trata de tudo | 🏆 Zero<br>GC                     | 🏆 Zero<br>Auto-Drop estático              |
| Fluidez do Modelo Arquitetural | 🟢 Alta<br>Faz tudo, perigosamente | 🔴 Baixa<br>Ownership rígida         | 🏆 Máxima                     | 🟢 Alta<br>multi-paradigma        | 🟡 Moderada<br>Alterna entre modelos       |
| Manipulação de Grafos Cíclicos | 🏆 Trivial<br>Ponteiros brutos     | 🔴 Restritiva<br>Rc<RefCell>         | 🏆 Inerente                   | 🏆 Inerente<br>GC                 | 🏆 Limpa<br>using / withRegion (§8)        |
| Manutenibilidade a Longo Prazo | 🔴 Frágil                          | 🏆 Excelente<br>Código sem medo      | 🟢 Alta                       | 🟢 Alta<br>tipos fortes           | 🏆 Excelente<br>Se compila, corre no metal |

## E. Bugs de Transbordo de Memória (Overflow)

|                                 | Linguagem C  | Rust          | Haskell Original | OCaml (modes)                   | Axión V0.2                              |
|---------------------------------|--------------|---------------|------------------|---------------------------------|-----------------------------------------|
| Stack: Thunks Pendentes         | — Não existe | — Não existe  | 🔴 Risco Alto     | 🟢 Raro<br>estrito (Lazy opt-in) | 🏆 Impossível<br>Strict por padrão       |
| Heap: Space Leaks (Preguiça)    | — Não existe | — Não existe  | 🔴 Risco Crítico  | 🟢 Raro<br>estrito               | 🏆 Impossível<br>Fim dos thunks livres   |
| Heap: Ciclos de Referência (Rc) | — Não existe | 🟡 Risco Médio | 🏆 Imune<br>GC    | 🏆 Imune<br>GC                   | 🏆 Seguros<br>Tokens Fantasma (§3D)      |
| Heap: Retenção por Arena        | — Não existe | — Não existe  | — Não existe     | — Não existe                    | 🟢 Segura<br>Escape = erro de tipo (§3C) |

## F. Curva de Aprendizagem (o «desafio» cognitivo)

|                       | Linguagem C | Rust         | Haskell Original | OCaml (modes)              | Axión V0.2                                     |
|-----------------------|-------------|--------------|------------------|----------------------------|------------------------------------------------|
| Barreira de Entrada   | 🟡 Moderada  | 🔴 «O Muro»   | 🟡 Moderada       | 🟡 Moderada<br>ML + módulos | 🟡 «O Muro, com vias»<br>Onboarding guiado (§8) |
| Carga Cognitiva       | 🔴 Alta      | 🔴 Alta       | 🟡 Média          | 🟡 Média<br>modes adicionam | 🟡 Alta (gerida)<br>Níveis L0-L3 + LSP (§8)     |
| Natureza do Erro      | 💣 Runtime   | 🔧 Compilação | 🌱 Lógica         | 🔧 Compilação               | 🧠 Topológica                                   |
| Suporte do Compilador | ◦ Neutro    | 🔧 Punitivo   | 🌱 Neutro         | 🌱 Neutro<br>merlin         | 💡 Sócrático                                    |

**Leitura honesta da tabela F:** a Axión troca a latência e as fugas de memória por carga cognitiva na entrada. É deliberadamente uma linguagem de nicho de alto desempenho — não um substituto de propósito geral para Python ou Go.

**G. Onde a Axión ainda perde (o reverso honesto)**

Um quadro em que a equipa da casa vence tudo não é honesto — e desconfia-se dele. Eis os eixos onde a Axión está, hoje, atrás de todos os concorrentes maduros:

| Aspeto                       | Axión           | Porquê / face a quem                                                                |
|------------------------------|-----------------|-------------------------------------------------------------------------------------|
| Ecosistema e bibliotecas     | ● Inexistente   | Sem Hackage nem crates; stdlib de raiz. C/Rust/OCaml têm décadas (§14 #03).         |
| Tooling (debugger, profiler) | ● Por construir | gdb/lldb, perf, valgrind não suportam Axión; o LSP nasce só na Fase 4 (§17).        |
| Maturidade / produção        | ● Zero          | É uma visão arquitetural, não um compilador. Rust/OCaml estão em produção há anos.  |
| Massa crítica / comunidade   | ● Nenhuma       | Dia 1, sem utilizadores. Linguagens morrem por falta disto (§14 #02, #08).          |
| Tamanho de binário / startup | ● Por medir     | Runtime próprio, sem stdlib partilhada. Alvo, não medição.                          |
| Portabilidade de features    | ● Limitada      | LLVM dá alvos, mas MPK/BMI2/CHERI restringem features avançadas a x86/ARM recentes. |

**Regra:** estes ● não são desculpas — são o mapa do que a Fase 1–4 (§17) tem de resolver antes de a Axión sair do papel. A aposta é que o *núcleo* (segurança + latência) justifica pagar tudo isto.

**Como estes quadros serão validados (metodologia)**

Os troféus acima são **desenho, não medição** — e é assim que se tornam falsificáveis. O plano de validação por eixo:

| Eixo                   | Baselines                | Métrica                                               | Limiar de sucesso                                                    |
|------------------------|--------------------------|-------------------------------------------------------|----------------------------------------------------------------------|
| Latência (jitter)      | C, Rust, Go, Haskell/GHC | p99 / p99.9 de pausa sob carga sustentada             | Sem pausas acima do ruído do SO; zero eventos de GC                  |
| Throughput numérico    | C (–O3), Rust            | GFLOP/s em kernels SIMD                               | Dentro de ~5% de C                                                   |
| Memória (RSS)          | C, Rust, Haskell         | pico e <i>high-water</i> de arena                     | ≈ C/Rust; sem retenção por arena após reset                          |
| Feedback do compilador | rust-analyzer, GHCI      | latência de recheagem por edição                      | Sub-milissegundo por módulo (Salsa)                                  |
| Ergonomia (carga)      | Rust equivalente         | LOC, nº de anotações, % de código L0/L1               | Menos anotações que Rust; ≥90% L0/L1                                 |
| Compactação ternária   | 2-bit, um-byte-por-enum  | <i>cache misses</i> / largura de banda (ML {–1,0,+1}) | Ganho líquido após custo de <i>unpack</i> , ou não se constrói (§10) |

**Regra de honestidade:** nenhuma célula de troféu vira alegação pública antes de passar o seu limiar num benchmark reproduzível. Onde o número não existir, o quadro deve dizer «por medir», não 🏆.



*O Fogo Amigo — as críticas mais duras que a Axión ouviria de engenheiros cépticos, e a resposta franca a cada uma. Um projeto só sobrevive se souber onde é frágil.*

#### 01 «Ninguém pediu uma linguagem ‘quântica’»

MITIGADA

**A objecção.** O rótulo «quântico» soa a *marketing* e afasta engenheiros de sistemas sérios.

**Resolvido (decisão de nome).** Renomeado: o termo normativo é «**topologia de tipos**», o *acoplamento por sessão* substituiu o «emaranhamento» e o `Trit` é um enum (não um «motor»). «Quântico» fica só como intuição didáctica pontual (§9). O mecanismo sempre foi conjunção aditiva + session types — agora o nome di-lo.

#### 02 A curva «K2» condena a adoção

MITIGADA

**A objecção.** Se o Rust já é «O Muro», uma carga cognitiva topológica ainda maior garante que quase ninguém sobe a montanha. Linguagens difíceis morrem por falta de massa crítica.

**A resposta da Axión.** Verdadeiro se tudo estiver sempre ligado — e por isso não está. A Ergonomia (§8) desliga a dificuldade por omissão em duas frentes. **Carga cognitiva:** níveis L0-L3 (a Polaridade Estática esconde a topologia até ser usada — 90% do código é L0/L1), compilador Socrático que mostra o grafo, e uma biblioteca de combinadores ( `pipeline` , `parMap` , `withRegion` ) que prova a topologia difícil uma vez.

**Barreira de entrada:** L0 lê-se como Haskell estrito (dia 1 sem conceitos novos), erros que ensinam, *playground* sem instalação e caminho guiado L0→L3. **Resultado:** a carga desce de *Extrema* a *Alta gerida* e a barreira de «O K2» a «O Muro, com vias» — ao nível do Rust, não pior (tabela F).

#### 03 Sem ecossistema, morre à nascença

VÁLIDA

**A objecção.** Uma linguagem sem bibliotecas, *package manager* maduro nem comunidade é um brinquedo académico. O Hackage e os *crates* demoraram uma década a existir.

**A resposta da Axión.** A resposta é assumida e sem atalhos: **não há compatibilidade com o Hackage** — as bibliotecas Haskell pressupõem laziness e GC, incompatíveis com o runtime de arenas — e a biblioteca padrão é desenhada de raiz, estrita e linear. A estratégia de ecossistema no dia 1 é a mesma do Zig e do Odin, com uma vantagem: a **FFI Linear Autogovernada** ( `@cleanup` ) reaproveita, com segurança governada pelo sistema de tipos, todo o universo de bibliotecas C e Rust já existentes, módulo a módulo, em vez de as reescrever. Do GHC herda-se a investigação (Linear Haskell, Stream Fusion) e o protótipo de validação da Fase 0 — a linhagem, não o ecossistema.

#### 04 A falácia do «compilador suficientemente inteligente»

PARCIALMENTE VÁLIDA

**A objecção.** Auto-Drop, Stream Fusion, defuncionalização e reset NLL prometem «custo zero mágico». Historicamente, otimizações que dependem do compilador ser esperto são **frágeis e imprevisíveis**.

**A resposta da Axión.** Diferença-chave: a Axión não *adivinha*, ela **prova**. As garantias vêm do sistema de tipos lineares (verificado, não heurístico) e as regiões de otimização são **explícitas** ( `imperative` , `using` , `bound` ). O programador sabe onde a fusão acontece — não depende da sorte do inliner.

#### 05 «Acoplamento» é uma data race com outro nome

MITIGADA

**A objecção.** Um efeito não-local que altera estado noutra thread é, por definição, exatamente aquilo que a promessa de «zero data races» devia proibir. Uma leitura ingénua do acoplamento parece contraditória.

**A resposta da Axión.** O efeito «acoplado» é mediado por um canal linear: `~` e `Maybe~` *desugar* para os endpoints da §06 (ver §9.D), a posse move-se e nunca é partilhada. «Zero data races» é, assim, um corolário da Fidelidade de Sessão, não uma promessa avulsa. A ausência de deadlocks vem do bloco `bound`, um *nursery* estruturado cuja topologia de comunicação é acíclica por construção.

#### 06 Anos-homem impossíveis para um núcleo minúsculo

VÁLIDA

**A objecção.** Salsa + tipos lineares + compactação ternária + fusão + backend nativo — e agora um compilador e runtime escritos de raiz — é o esforço de engenharia de uma década e de uma equipa: risco enorme de nunca sair do papel.

**A resposta da Axión.** Por isso o roteiro é faseado e cada fase entrega valor isolado: a Fase 0 valida a semântica num protótipo EDSL descartável sobre o Linear Haskell — aprende-se barato antes do grande build — e cada fase do compilador próprio entrega valor isolado. As camadas de topologia avançada e de compactação ternária são opcionais e desligadas por omissão — a linguagem é útil antes de estar completa.

#### 07 O ternário balanceado não compensa em hardware binário

MITIGADA

**A objecção.** Emular trits com 2 bits, mais o custo de packing/unboxing base-243, pode comer o ganho branchless. Sem números reais, é otimização especulativa.

**A resposta da Axión.** Válida contra um *motor* ternário pervasivo — e por isso ele deixou de existir. O ternário foi **demovido** a duas peças separadas (§10): o tipo `Trit` é apenas um **enum de três estados** (custo zero, como o `Ordering` do Rust) para `observe` /Kleene, e a seleção branchless não é sequer específica dele. A **compactação base-243** — a única parte que emula ternário — passou para uma **biblioteca opcional** (`TritVec`), desligada por omissão, dirigida ao caso onde a densidade paga mesmo (**ML quantizado ternário**, pesos  $\{-1,0,+1\}$ ) e validada por benchmark contra baselines de 2 bits e de um-byte-por-enum. Onde não paga, não existe — logo não há custo especulativo a comer ganho nenhum.

#### 08 Mercado inexistente: quem migra?

PARCIALMENTE VÁLIDA

**A objecção.** Ninguém reescreve um sistema em produção para uma linguagem nova de risco. O custo de migração esmaga qualquer ganho teórico.

**A resposta da Axión.** A Axión não disputa o mercado geral. O alvo são nichos onde o jitter do GC é inaceitável e a segurança é crítica: HFT, kernels, motores de crypto, sistemas de tempo-real e drivers. Aí o valor é concreto e a FFI permite migração incremental, módulo a módulo.

### 09 «O Boxed é um garbage collector envergonhado»

MITIGADA

**A objecção.** Se a linguagem precisa de *reference counting* para os casos difíceis, a promessa «sem GC» é marketing: RC é gestão automática de memória, com custos escondidos nos incr/decr e nas cascatas de drop.

**A resposta da Axión.** RC determinístico não é tracing GC: não há pausas, não há *heap scans*, e cada decref acontece num ponto estático conhecido — o jitter zero mantém-se. O `Boxed` é opt-in, visível no tipo, restrito a L1+ e vigiado por lint (§3); a análise de linearidade elide a maioria dos contadores, e a imutabilidade elimina os ciclos — logo não há coletor de ciclos. A cascata  $O(n)$  do drop é assumida na spec, não escondida, e o subsistema só se constrói se a telemetria (AX0003) o justificar. O caso por omissão continua a ser arenas com custo zero.

### 10 Tarefas leves sem GC é uma fantasia

PARCIALMENTE VÁLIDA

**A objecção.** O Go e o GHC só têm milhões de green threads porque o GC lhes move as stacks. Sem GC, ou se pagam threads do SO, ou reaparece o `async/await` com o seu *function coloring*. Prometer as duas coisas ao mesmo tempo é fantasia.

**A resposta da Axión.** A Axión não usa stacks móveis: as tarefas são continuações defuncionalizadas — máquinas de estado — alocadas na arena do nursery (§11), e os pontos de suspensão são exatamente as operações de canal, já visíveis nos tipos (`Bound s`), pelo que não há `async/await` na superfície nem coloring. A recursão através de suspensões vai para frames CPS na arena; `imperative` nunca suspende (§9.C). O risco que resta é de engenharia, não de conceito — por isso o runtime é entregável da Fase 3, com benchmarks de escalabilidade antes de qualquer promessa numérica.

### 11 «Já existe OCaml com *modes* — porquê Axión?»

PARCIALMENTE VÁLIDA

**A objecção.** O trabalho de *modes* do OCaml («Oxidizing OCaml») já dá uniqueness/linearidade inferida e alocação em stack sem lifetimes — a tese central da Axión. Reinventar uma linguagem inteira para isso é desperdício.

**A resposta da Axión.** Objecção justa — é o concorrente mais próximo (§16), e a Axión não reclama ineditismo na peça isolada. A diferença é de **combinação e ponto de partida**: os *modes* do OCaml acrescentam-se a um runtime **com GC**, ao passo que a Axión é sem-GC *por construção*; e integra *session types* lineares, concorrência estruturada e divulgação progressiva L0-L3 no núcleo, não como extensões. Se a necessidade é «OCaml um pouco mais linear», usa-se OCaml. Se é «latência zero garantida + concorrência sem data races por tipos», é aí que a Axión se justifica.

**Veredicto.** Das onze objecções, quatro são de comunicação e mercado/posicionamento (mitigáveis com comunicação) e sete são de engenharia — todas com defesa técnica concreta. As mais sérias — o acoplamento por sessão (05) e as tarefas leves sem GC (10) — têm resposta operacional na §9.D e na §11, e ambas ficam sob prova: metateoria mecanizada para uma, benchmarks de escalabilidade para a outra. Uma visão que sobrevive ao seu próprio fogo amigo está pronta para começar a ser construída.

| Área                                                            | Estado   | Risco / Trabalho futuro                                                                                                                                                                                                                                                                                     |
|-----------------------------------------------------------------|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Tipos lineares + Auto-Drop                                      | Sólido   | Ergonomia da inferência de regiões em código altamente polimórfico.                                                                                                                                                                                                                                         |
| Regra Drop (destruidor como consumidor)                         | Definido | Must-use vs droppable (§2); endpoints cancelam em pânico, não se descartam (§7); formalizar na Fase 3 ( <i>Exceptional GV</i> ).                                                                                                                                                                            |
| Arenas NLL                                                      | Sólido   | Escape de sub-arena é erro de compilação ( <code>promote</code> ); marcas de arena e <i>decommit</i> no reset reclamam a memória residual.                                                                                                                                                                  |
| Session Types                                                   | Sólido   | Race-freedom por Fidelidade de Sessão e deadlock-freedom pelo <code>bound</code> estruturado; resta afinar a inferência de protocolos e mensagens de erro legíveis.                                                                                                                                         |
| Runtime (scheduler M:N)                                         | Definido | Tarefas defuncionalizadas em arenas de nursery (§11); cancelamento por ramo de protocolo (§7); implementação na Fase 3, com benchmarks de escalabilidade.                                                                                                                                                   |
| FFI (isolamento real)                                           | Definido | Isolamento em camadas opt-in, declarado no <code>foreign</code> (§5): <code>@trusted</code> (arena, confina fugas) → <code>@isolate(mpk)</code> → <code>@sandbox(wasm)</code> → CHERI. Implementação faseada; a arena confina bugs, MPK/WASM confinam ataques.                                              |
| <code>Boxed a</code> (RC Perceus)                               | Definido | Escape-hatch opt-in com RC determinístico (§3); sem coletor de ciclos (imutável); pós-Fase 2, condicionado à telemetria de erros (§8).                                                                                                                                                                      |
| Tipo <code>Trit</code> (observe/Kleene)                         | Sólido   | Enum de três estados, caso $N=3$ dos sum types; seleção branchless geral de enums pequenos. Custo zero, sem maquinaria dedicada (§10).                                                                                                                                                                      |
| Compactação base-243 ( <code>TritVec</code> )                   | Opcional | Biblioteca, não núcleo; desligada por omissão. Ganho depende da microarquitetura e do workload (ML ternário); medir vs. 2-bit e vs. um-byte-por-enum antes de investir (§14 #07).                                                                                                                           |
| Acoplamento por sessão ( <code>~</code> / <code>Maybe~</code> ) | Parcial  | Semântica operacional na §9.D; deadlock-freedom por construção ( <code>bound</code> estruturado). Metateoria fundada em lógica linear ( <i>Propositions as Sessions</i> ), a mecanizar (Iris/Actris) e a validar por <b>verificador de referência em compilação</b> — sem monitores em runtime, custo zero. |
| Permissões fracionárias ( <code>%0.5</code> )                   | Definido | Split/join ao estilo de Boyland (§2): leitura partilhada em <code>%0.5</code> , escrita recuperada no <i>join</i> para <code>%1</code> .                                                                                                                                                                    |
| Ergonomia / carga cognitiva                                     | Parcial  | Níveis L0-L3, compilador Sócrático e biblioteca de combinadores (§8) tornam a carga topológica paga-quando-se-usa; a implementação do LSP e da biblioteca é trabalho de engenharia.                                                                                                                         |
| Firewall de níveis ( <code>LEVEL</code> )                       | Definido | Pragma <code>{-# LEVEL #-}</code> + <code>max-level</code> no manifesto (§8); custo ~zero sobre a Polaridade; entra com o LSP (Fase 4).                                                                                                                                                                     |
| Diagnósticos AXnnnn                                             | Em curso | Códigos estáveis, JSON e fixes <i>machine-applicable</i> (§8); o registo nasce na Fase 1; a telemetria decide o <code>Boxed</code> (§3).                                                                                                                                                                    |

## Roteiro sugerido — numeração canônica e detalhe completo na §17

- 1 **Fase 0 — Fundações:** protótipo EDSL descartável sobre `LinearTypes` (bancada de validação semântica, não o produto), repositório, CI e subconjunto mínimo.
- 2 **Fase 1 — Núcleo linear (L0/L1):** esqueleto ambulante (*parse* → *typecheck* → *correr*); Auto-Drop e linearidade no compilador próprio. Sem topologia avançada.
- 3 **Fase 2 — Modelo de memória:** arenas NLL, escape via `promote`, permissões fracionárias e FFI; benchmark vs C/Rust.
- 4 **Fase 3 — Concorrência (+ trilho formal):** canais + session types com prova de ausência de data races e deadlock-freedom estrutural.
- 5 **Fase 4 — Ergonomia e tooling:** LSP, diagnósticos `AXnnnn`, playground e níveis L0–L3.
- 6 **Fase 5a — Compactação ternária (biblioteca):** o tipo `Trit` (enum) existe desde a Fase 1; o entregável aqui é a biblioteca `TritVec` (packing base-243 + kernels SIMD) — só se o benchmark o justificar, com alvo em ML ternário.
- 7 **Fase 5b — Topologia de tipos avançada:** o acoplamento (`~` / `Maybe~`) tem semântica operacional na §9.D e deadlock-freedom por construção via `bound` estruturado; falta **mecanizar** a metateoria (via *Propositions as Sessions*, em Iris/Actris), **extrair o verificador de referência** em compilação (custo zero, sem monitores em runtime) e dar semântica denotacional a `&` / `&?` antes de estabilizar a API.

Esta especificação (V0.2) fixa a **visão arquitetural** da Axióon — legível, ilustrada e implementável por fases — convertendo as abstrações da lógica linear e ternária balanceada em instruções de máquina puras, *branchless* e de custo nulo sobre hardware clássico.

*A Axióon não inventa fundamentos: combina linhagens estabelecidas. Estas são as obras que sustentam cada mecanismo.*

## Tipos lineares e de unicidade

- JY. Girard. *Linear Logic*. Theoretical Computer Science, 1987.
- E. Barendsen, S. Smetsers. *Uniqueness Typing for Functional Languages* (Clean). MSCS, 1996.
- JP. Bernardy, M. Boespflug, R. Newton, S. Peyton Jones, A. Spiwack. *Linear Haskell*. POPL 2018.
- D. Orchard, VB. Liepelt, H. Eades III. *Quantitative Program Reasoning with Graded Modal Types* (Granule). ICFP 2019.

## Memória por regiões (sem GC)

- M. Tofte, JP. Talpin. *Region-Based Memory Management*. Information and Computation, 1997.
- D. Grossman et al. *Region-Based Memory Management in Cyclone*. PLDI 2002.
- J. Boyland. *Checking Interference with Fractional Permissions*. SAS 2003.
- A. Reinking, N. Xie, L. de Moura, D. Leijen. *Perceus: Garbage Free Reference Counting with Reuse*. PLDI 2021.
- S. Ullrich, L. de Moura. *Counting Immutable Beans: Reference Counting Optimized for Purely Functional Programming* (Lean). IFL 2019.
- EZ. Yang, G. Campagna, Ö. Ağacan et al. *Efficient Communication and Collection with Compact Normal Forms* (GHC Compact Regions). ICFP 2015.
- E. Ovadia. *Vale — region borrowing e generational references* (linguagem).

## Session types e concorrência

- K. Honda, V. Vasconcelos, M. Kubo. *Language Primitives and Type Discipline for Structured Communication-Based Programming*. ESOP 1998.
- K. Honda, N. Yoshida, M. Carbone. *Multiparty Asynchronous Session Types*. POPL 2008.
- PM. Deniérou, N. Yoshida. *Multiparty Session Types Meet Communicating Automata*. ESOP 2012.

## Proposições como sessões e deadlock-freedom

- L. Caires, F. Pfenning. *Session Types as Intuitionistic Linear Propositions*. CONCUR 2010.
- P. Wadler. *Propositions as Sessions*. ICFP 2012.
- N. Kobayashi. *A Type System for Lock-Free Processes*. Information and Computation, 2002.
- L. Padovani. *Deadlock and Lock Freedom in the Linear  $\pi$ -Calculus*. CSL-LICS 2014.
- A. Igarashi, P. Thiemann, V. Vasconcelos, P. Wadler. *Gradual Session Types*. ICFP 2017.
- S. Fowler, S. Lindley, J.G. Morris, S. Decova. *Exceptional Asynchronous Session Types: Session Types without Tiers*. POPL 2019.

## Verificação mecanizada

- R. Jung et al. *Iris from the Ground Up*. JFP 2018.
- JK. Hinrichsen, J. Bengtson, R. Krebbers. *Actris: Session-Type Based Reasoning in Separation Logic*. POPL 2020.

- R. Jung, JH. Jourdan, R. Krebbers, D. Dreyer. *RustBelt: Securing the Foundations of the Rust Programming Language*. POPL 2018.
- A. Pnueli, M. Siegel, E. Singerman. *Translation Validation*. TACAS 1998.
- X. Leroy. *Formal Verification of a Realistic Compiler* (CompCert). CACM 2009.

## Otimização, incrementalidade e ternário

- D. Coutts, R. Leshchinskiy, D. Stewart. *Stream Fusion: From Lists to Streams to Nothing at All*. ICFP 2007.
- JC. Reynolds. *Definitional Interpreters for Higher-Order Programming Languages* (defuncionalização). 1972.
- DE. Knuth. *The Art of Computer Programming, Vol. 2* (ternário balanceado; computador Setun, 1958).
- Salsa / rust-analyzer — computação incremental orientada a consultas (N. Matsakis et al.).

## Linearidade sem lifetimes — sistemas próximos (posicionamento)

- A. Lorenzen et al. *Oxidizing OCaml with Modal Memory Management* — modos de uniqueness/linearidade e alocação em stack, inferidos, sem lifetimes. ICFP 2024 (e a série «Oxidizing OCaml» da Jane Street).
- F. Borretti. *Austral* — linguagem de sistemas com tipos lineares e capacidades; linearidade minimalista (spec).
- D. Racordon et al. *Implementation Strategies for Mutable Value Semantics* (Hylo / Val). JOT / Programming 2022.

**Posicionamento honesto.** A tese central da Axió — *linearidade e regiões inferidas, sem lifetimes explícitos* — **não é única**. O concorrente vivo mais próximo são os **modos do OCaml** («Oxidizing OCaml»): uniqueness/linearidade e *stack allocation* por *modes* inferidos, sem as anotações de tempo de vida do Rust. A Axió distingue-se por **combinação**, não por exclusividade: (1) é estrita e **sem GC por construção** — o OCaml mantém o runtime com GC; (2) integra *session types* lineares e concorrência estruturada no núcleo; (3) aposta na divulgação progressiva L0-L3. **Austral** é o melhor prior art de ergonomia linear minimalista; **Granule** fundamenta as multiplicidades graduadas ( `%1 / %0.5` ); **Vale** e **Hylo** exploram regiões e semântica de valores mutáveis — parente das atualizações destrutivas (§2/§4).

## Isolamento e sandboxing (FFI)

- S. Narayan, C. Disselkoen, T. Garfinkel et al. *Retrofitting Fine Grain Isolation in the Firefox Renderer* (RLBox / wasm2c). USENIX Security 2020.
- A. Vahldiek-Oberwagner, E. Elnikety, N. Duarte et al. *ERIM: Secure, Efficient In-process Isolation with Protection Keys* (MPK). USENIX Security 2019.
- RNM. Watson, J. Woodruff, PG. Neumann et al. *CHERI: A Hybrid Capability-System Architecture for Scalable Software Compartmentalization*. IEEE S&P 2015.



## 17 Plano de Implementação (do Início ao Fim)

O ativo real da Axióon é o núcleo linear-funcional; a compactação ternária ( `TritVec` ) e a topologia de tipos avançada vêm no fim, atrás de um interruptor. O plano é **faseado para que cada etapa entregue valor sozinha** — se parar a meio, resta à mesma uma linguagem de sistemas útil.

### Princípios de execução

- Esqueleto ambulante primeiro.** A Fase 1 não entrega uma feature, mas *parse* → *typecheck* → *correr* de ponta a ponta num subconjunto mínimo. Tudo cresce daí.
- Núcleo antes da plumagem.** Linear + memória + concorrência antes de ternário/topologia avançada. Se o projeto parar a meio, resta um «Rust funcional» útil.
- Provar antes de construir (onde é barato).** Para a concorrência, formalizar o cálculo *antes* de o implementar — a teoria dita o desenho certo.
- Prototipar no GHC, construir de raiz.** O Linear Haskell é a bancada de validação semântica da Fase 0; o compilador, o backend e o runtime da Axióon são escritos de novo — o GC e a laziness do GHC são incompatíveis com as garantias da linguagem.
- Testes e *dogfooding* desde o dia 1.** Property-based tests a acompanhar cada regra; escrever exemplos reais na própria Axióon à medida que cresce.

### Roteiro em relance

| Fase | Foco                                          | Duração         | Entrega isolada                                     |
|------|-----------------------------------------------|-----------------|-----------------------------------------------------|
| 0    | Decisões e fundações                          | 1-2 sem         | Estratégia, repo, subconjunto mínimo                |
| 1    | Esqueleto ambulante (L0/L1)                   | 1-3 meses       | Compilador mínimo: linearidade end-to-end           |
| 2    | Modelo de memória                             | 3-6 meses       | Segurança de memória sem GC                         |
| 3    | Concorrência + trilha formal                  | 4-8 meses       | Race- e deadlock-freedom                            |
| 4    | Ergonomia e tooling                           | contínuo        | Usabilidade: LSP, erros, playground                 |
| 5a   | Compactação ternária ( <code>TritVec</code> ) | após núcleo     | base-243 / codecs SIMD (medido)                     |
| 5b   | Topologia de tipos avançada                   | após metateoria | <code>~</code> / <code>Maybe~</code> (experimental) |

### Fase 0 — Decisões e fundações (1-2 semanas)

O quê: fixar âmbito e infraestrutura antes de uma linha de compilador.

- Decisão-chave (estratégia):** começar como EDSL sobre Linear Haskell ( `LinearTypes` + `linear-base` ) para validar a semântica em semanas — um **protótipo descartável, não o embrião do compilador**; migrar depois para o compilador autónomo (frontend próprio → Axióon Core, IR linear estrito → backend nativo). De-risca: aprende-se barato antes do grande build. **Limites da bancada:** o EDSL valida o núcleo `%1` (L0/L1); `%0.5` , `&` , `~` e os session types não são exprimíveis no `LinearTypes` do GHC (multiplicidades só nas setas) — validam-se no trilha formal da Fase 3 e no typechecker próprio.
- `git init` , projeto Cabal (protótipo), CI, formatador, linter.
- Escrever a gramática mínima (subconjunto L0/L1) e os programas-alvo que definem o sucesso da Fase 1.
- A spec (este documento) passa a estar versionada ao lado do código.



## | Fase 1 — Esqueleto ambulante (núcleo L0/L1) (1–3 meses)

O quê: o compilador mínimo que corre um programa linear.

- Parser do subconjunto mínimo (tipos, funções, `let`, `%1`).
- Registo de **códigos de erro estáveis** + diagnósticos estruturados (JSON) desde o primeiro erro emitido — retrofitar um registo depois de existirem centenas de erros é doloroso (§8).
- Typechecker próprio com linearidade — validado por diferencial contra o protótipo EDSL da Fase 0 (§2).
- Baixar para um backend: interpretador próprio (o futuro fast-path de `--dev`), com o backend LLVM como alvo da fase seguinte.
- **Meta:** a Listagem 2.1 compila e corre; um uso-após-consumo é rejeitado. Property tests a verificar preservação/progresso.

## | Fase 2 — Modelo de memória (o diferenciador) (3–6 meses)

O quê: o que torna a Axióon ≠ «só mais um Haskell».

- **Auto-Drop** — análise de liveness que injeta `free` no ponto de morte (§2).
- **Arenas + reset NLL + análise de escape** (`promote`, §3) — validar que o escape é erro de compilação.
- **Permissões fracionárias** (`%0.5`): `split` / `join`.
- **Benchmark contra baseline (C/Rust)** — a «latência zero» tem de ser medida, não afirmada.

## | Fase 3 — Concorrência (canais + session types) (4–8 meses)

O quê: race-freedom e deadlock-freedom.

- **Trilho formal** (começa antes do código): formalizar o cálculo via *Propositions as Sessions* — incluindo o cancelamento em pânico (*Exceptional Asynchronous Session Types*, §7); a eliminação de cortes dá o desenho correto do `bound`.
- Implementar canais lineares + session types + `bound` (nursery, topologia acíclica, §9).
- Implementar o runtime da §11: scheduler M:N, tarefas defuncionalizadas em arenas de nursery, pool dedicado para FFI bloqueante.
- **Validar:** property tests + model-checking (CFSMs) → metateoria mecanizada (Iris/Actris) como objetivo de médio prazo.

## | Fase 4 — Ergonomia e tooling (decide a adoção) (contínuo, começa cedo)

O quê: transformar «poderoso» em «usável» — a barreira K2→Muro (§8).

- **Mensagens de erro que ensinam** (o compilador Socrático) — investir aqui mais do que parece razoável; foi o que salvou o Rust. Com códigos estáveis (`--explain`), diagnósticos JSON e fixes *machine-applicable* (§8); a telemetria de erros daqui decide o `Boxed` (§3).
- Níveis L0–L3 impostos pela Polaridade Estática; teto por pragma `{-# LEVEL ... #-}` e `max-level` no manifesto (§8).
- LSP + playground em browser (sobre o Salsa); `axion new`, exemplos «Axióon by Example».

## | Fase 5a — Compactação ternária, biblioteca `TritVec` (atrás de um interruptor)

O quê: o tipo `Trit` já é um enum disponível desde a Fase 1; o entregável é a **biblioteca opcional** `TritVec` — packing base-243 + codecs SIMD (§10), desligada por omissão. **Como:** implementar e medir se o ganho existe mesmo (largura de banda vs. custo de *unpack*, em BMI2/ARM) contra baselines de 2 bits e de um-byte-por-enum, com alvo em ML quantizado ternário, antes de investir.

## | Fase 5b — Topologia de tipos avançada (a última, e só se ganhar o direito)

**O quê:** ~ / Maybe~ (§9). **Como:** só depois da metateoria mecanizada + verificador de referência em compilação (custo zero, §9.D). Manter marcado *experimental* até a prova aterrar — é a peça mais memorável e a menos urgente.

## | Transversais (a correr o tempo todo)

- FFI cedo ( `LinearPtr` / `@cleanup` , §5) — reaproveitar bibliotecas C/Rust para não morrer sem ecossistema.
- Benchmarks contínuos, testes por propriedades, e a spec versionada junto ao código.

## | As primeiras 2 semanas

- 1 `git init` + projeto Cabal do protótipo EDSL com `LinearTypes` ligado (o repositório do compilador próprio nasce na Fase 1).
- 2 Escrever 5 programas-alvo Axióon que definem «sucesso da Fase 1».
- 3 Protótipo EDSL: um `Buffer %1` que o typechecker recusa usar duas vezes — a semântica de linearidade validada antes de se escrever o compilador.
- 4 Montar CI + a estrutura de property tests (o andaime, não os testes ainda).

**Nota honesta de esforço:** isto é, realisticamente, um esforço de anos — mais ainda com compilador e runtime próprios — mas faseado para dar valor a cada passo. No fim da Fase 2 já existe algo publicável e útil (memória segura sem GC), muito antes de tocar na topologia avançada. A ordem protege o projeto: cada fase entrega sozinha.

## 18 Decisões de Implementação (Stack e Auto-Hospedagem)

A §8 fixa a arquitetura (motor de consultas Salsa) e a §11 o pipeline (Cranelfift em `--dev`, LLVM em `--release`); esta secção fecha o como concreto que faltava — a linguagem de implementação e os crates — e declara a intenção de longo prazo: **reescrever tudo na própria Axióon**.

### Três camadas, uma regra

A toolchain separa-se em três camadas com pares diretos noutras linguagens. A estratégia de construção decorre de um princípio único: **forkar nada; escrever de raiz o que carrega a semântica da linguagem; reutilizar como biblioteca o que é genérico e sem semântica**.

| Camada                                           | Par (Rust / Haskell)  | Estratégia                                                                                                                                                                                                                       |
|--------------------------------------------------|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Driver / gestor de pacotes<br><code>axion</code> | cargo / cabal         | <b>De raiz</b> , imitando o <i>modelo</i> do cargo: CLI única + manifesto <code>axion.toml</code> (com <code>max-level</code> , §8). Forkar o cargo daria mais trabalho — está acoplado a crates e à CLI do <code>rustc</code> . |
| Compilador<br><code>axionc</code>                | rustc / GHC           | <b>De raiz, em Rust</b> . Não forkar nenhum: são enormes e semanticamente opostos — o GHC é <i>lazy</i> +GC (§17.4), o <code>rustc</code> traz o seu modelo de crates e IR próprios.                                             |
| Motor incremental<br>Salsa                       | salsa (rust-analyzer) | <b>Reutilizar o <i>crate</i></b> como dependência. É substrato genérico, sem semântica de linguagem; não se reescreve nem se forka — constrói-se por cima, como o rust-analyzer.                                                 |

**Porquê Rust para o `axionc`**. O documento já nomeia Salsa, rust-analyzer, Cranelfift e LLVM — todos do ecossistema Rust. Escrever o compilador em Rust não é uma escolha nova: é herdar diretamente a linhagem do rust-analyzer para que os requisitos de LSP, incrementalidade e diagnósticos venham resolvidos de fábrica, em vez de reimplementados.

### Stack concreta em Rust — cada requisito, um crate maduro

| Requisito (secção)                                             | Crate                                            | Papel                                                                                                                                                                                    |
|----------------------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Motor de consultas incremental (§8)                            | <code>salsa</code>                               | Invalidação por nó, sub-milissegundo; o mesmo motor do rust-analyzer. Usar a geração atual (salsa 2022+), não a API clássica.                                                            |
| Análise léxica                                                 | <code>logos</code>                               | Lexer gerado por macro, muito rápido, para o <code>.axi</code> .                                                                                                                         |
| Árvore sintática <i>lossless</i>                               | <code>rowan</code>                               | CST resiliente a erros (do rust-analyzer) — essencial para o LSP e para o compilador Sócrático (§8). Emparelha com o Salsa: o <i>parse</i> é uma consulta memoizada sobre o texto-fonte. |
| Servidor LSP (grafo inline, §8)                                | <code>lsp-server</code> / <code>tower-lsp</code> | <code>lsp-server</code> é o que o rust-analyzer usa.                                                                                                                                     |
| Diagnósticos <code>AXnnnn</code> + <code>--explain</code> (§8) | <code>annotate-snippets</code>                   | Render de erros com spans/setas — a biblioteca extraída do <code>rustc</code> . Emissão JSON e fixes <i>machine-applicable</i> por cima.                                                 |
| Backend <code>--dev</code> (§11)                               | <code>cranelfift</code>                          | Codegen desenhado para compilar depressa — o «Fast-Path Backend» da tabela C.                                                                                                            |

Backend `--release`  
(§11)

LLVM via  
`inkwell`

O caminho otimizado (Assembly branchless BMI2/ARM), ligado só em *release*.

**Coerência com o roteiro.** A Fase 0 mantém-se um protótipo EDSL em Haskell/Cabal (validação semântica, §17), descartável; o `axionc` real em Rust nasce na Fase 1. As escolhas acima são a camada de implementação que assenta por baixo da arquitetura já fixada — não a alteram.

## Auto-hospedagem — o objetivo de longo prazo

A intenção final é reescrever toda a toolchain na própria *Axión*. A implementação em Rust é o **compilador de arranque (Stage 0)**, não o destino. Quando a linguagem amadurecer o suficiente — depois do núcleo linear, do modelo de memória sem GC, da FFI (§5) e do *tooling* (§17, Fases 1–4) — o `axionc`, o `axion` e o runtime são reescritos **em *Axión***: usa-se a versão Rust para compilar a primeira versão em *Axión* (Stage 1) e, a partir daí, o compilador compila-se a si próprio (Stage 2, *fixpoint*).

Isto é o *dogfooding* levado ao limite (§17.5): as garantias da linguagem — linearidade, Auto-Drop, ausência de GC, latência zero — validadas pela prova mais dura de todas, o compilador a compilar-se a si mesmo. O Rust é o andaime que se retira quando a *Axión* se sustém sozinha; até lá, permanece o *bootstrap* estável.

**Nota de honestidade.** A auto-hospedagem é trabalho de longo prazo, posterior às Fases 1–4 — depende de a *Axión* já ter FFI robusta e maturidade de biblioteca para exprimir um compilador. É a direção declarada, não um marco do roteiro imediato.

## 19 Exemplo Completo: um Módulo de Ponta a Ponta

As peças isoladas das secções anteriores, compostas num módulo realista — o texto que um programador escreveria de facto. Processa um fluxo de pacotes cifrados, num pipeline concorrente, sem GC, sem fugas e à prova de pânico.

Listagem 19.1 – Processador de pacotes: FFI + arena + stream + canal + unwinding

```
{-# LEVEL L2 #-} -- este módulo usa concorrência (canais); a firewall permite L2 (§8)
module PacketPipe (runPipe) where

-- FFI: contexto OpenSSL com limpeza automática (§5). O @cleanup é o Drop do handle.
foreign import ccall "ssl_ctx_new"  ssl_new  :: IO (LinearPtr SSL_CTX)
foreign import ccall "ssl_ctx_free" @cleanup freeCtx :: LinearPtr SSL_CTX -> IO ()

-- Um estágio decifra um pacote LENDO (borrow, §2) o contexto partilhado – não o consome.
decrypt :: SSL_CTX -> Packet %1 -> Packet %1
decrypt ctx pkt = imperative $ do -- região imperativa, fronteiras puras (§4)
    ssl_decrypt_inplace ctx pkt -- muta o buffer no sítio (destructive update)
    pkt

-- O pipeline corre num nursery 'bound': topologia acíclica => deadlock-free (§6, §9).
runPipe :: Arena %1 -> LinearStream Packet %1 -> IO ()
runPipe arena source = withResource (Acquire ssl_new) $ \ctx -> -- fecha mesmo em pânico
    bound arena $ do -- concorrência estruturada; canais confinados
        source
        |> stage (\p -> decrypt ctx p) -- estágio 1 (thread própria)
        |> stage validate -- estágio 2 (canal linear entre estágios)
        |> sink emit -- consome; a posse termina aqui
    -- Se 'validate' entrar em pânico: Linear Unwinding (§7) recua a arena numa
    -- instrução (os buffers evaporam, O(1)), dispara freeCtx(ctx), e envia o
    -- sinal de cancelamento (Closed) aos endpoints filhos. Zero recursos órfãos.
```

O que este módulo demonstra, tudo junto: FFI linear com limpeza injetada (§5), leitura por empréstimo sem consumir (§2), mutação in-place numa região imperativa (§4), *Stream Fusion* a colapsar o pipeline num único loop (§3B), canais com session types confinados a um *nursery* acíclico (§6, §9) e recuperação de pânico O(1) sem fugas (§7). **Nenhuma anotação de *lifetime*, nenhum `free` manual, nenhum GC** — e o nível L2 é imposto pela *firewall* (§8). É a tese da Axióm numa página de código.